

PROGETTO DI RETI DI CALCOLATORI E LABORATORIO

Università Parthenope di Napoli Parthenope

Progetto

Green Pass 



Prof. Alessio Ferone

Lavoro a cura di:

Francesco Mabilia
Matricola: 0124001910

&

Roberto Vecchio
Matricola: 0124001871

&

Gaetano Ippolito
Matricola: 0124001867

Indice

1.0 - Descrizione del progetto	3
2.0 - Descrizione e schemi dell'architettura	9
3.0 - Descrizione e schemi del protocollo applicazione	12
4.0 - Dettagli implementativi dei client	28
5.0 - Dettagli implementativi dei Server	35
6.0 - Manuale utente	37

1.0 - Descrizione del progetto

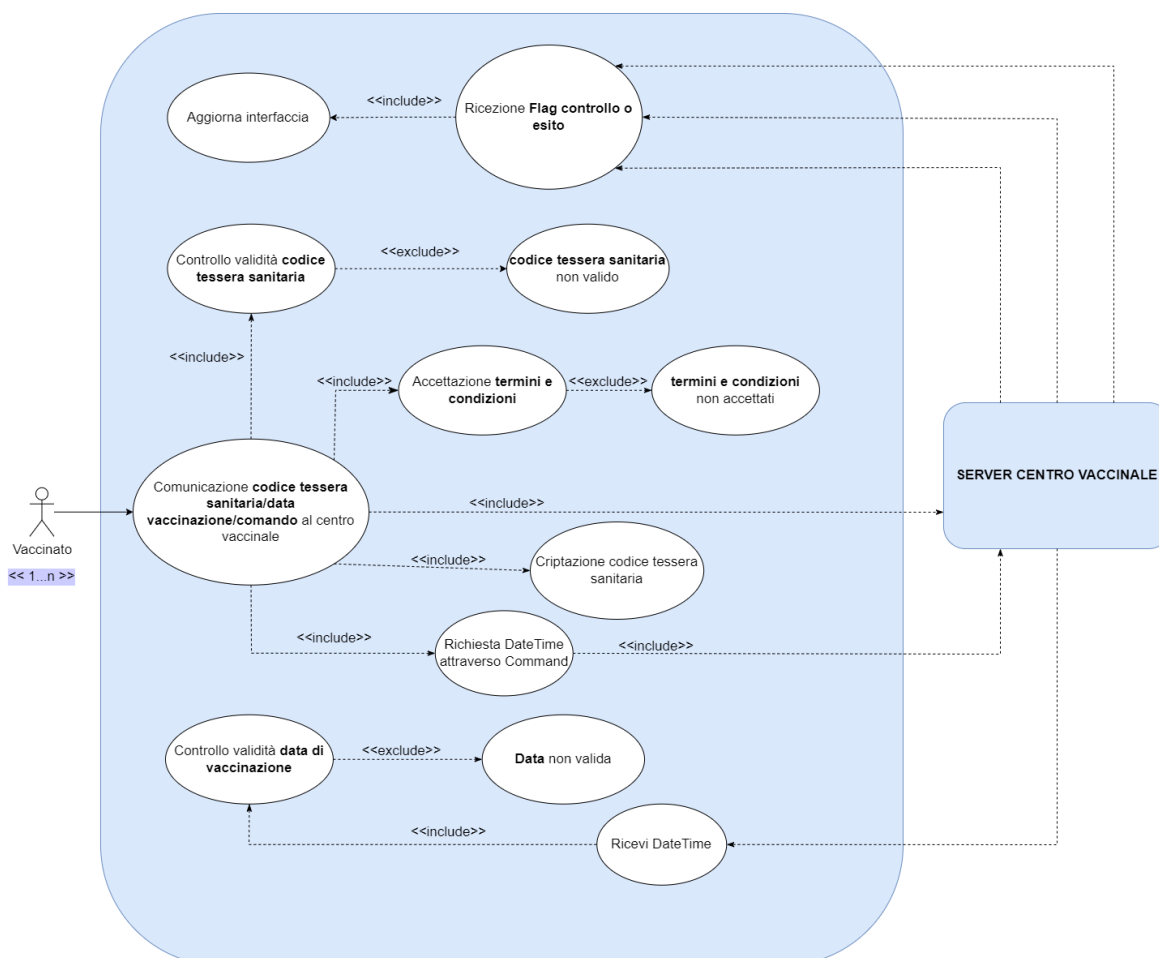
Nel presente progetto si vuole realizzare un sistema di gestione per la documentazione **Green Pass**. Per rilevare le entità che interagiscono con il sistema, si è proceduto con il realizzare un diagramma dei casi d'uso, il quale ha permesso di identificare, oltre alle entità, le tipologie di interazione che ognuno di essi potrà avere con il sistema e i sottosistemi.

Il sistema è costituito dalle seguenti entità:

- **Client (vaccinato):** Il client è connesso solo con il server centro vaccinale e attraverso il corrispettivo menu potrà iscriversi alla piattaforma **Green Pass**, inserendo un codice di tessera sanitaria valido ed una data di vaccinazione. In particolare il client eseguirà le seguenti azioni:

1. **Richiesta daytime:** per effettuare un'iscrizione, il client dovrà richiedere, automaticamente e senza interazione dell'utente, al server centro vaccinale data ed ora locale, in modo da permettere all'utente che interagisce con il client, l'inserimento di una data di vaccinazione non superiore alla data odierna e non inferiore di un mese rispetto a quest'ultima, come indicato nel protocollo livello applicazione ([capitolo 3](#)).
2. **Iscrizione alla piattaforma **Green Pass**:** per iscriversi alla piattaforma **Green Pass**, il client invierà un codice di tessera sanitaria valido ed un data di vaccinazione valida, come descritto dal punto 1. Una volta effettuata la registrazione, il client viene notificato relativamente dell'esito dell'operazione.

Di seguito mostriamo il diagramma dei casi d'uso da cui si sono ricavate le informazioni elencate:



- In particolare il server eseguirà le seguenti azioni:

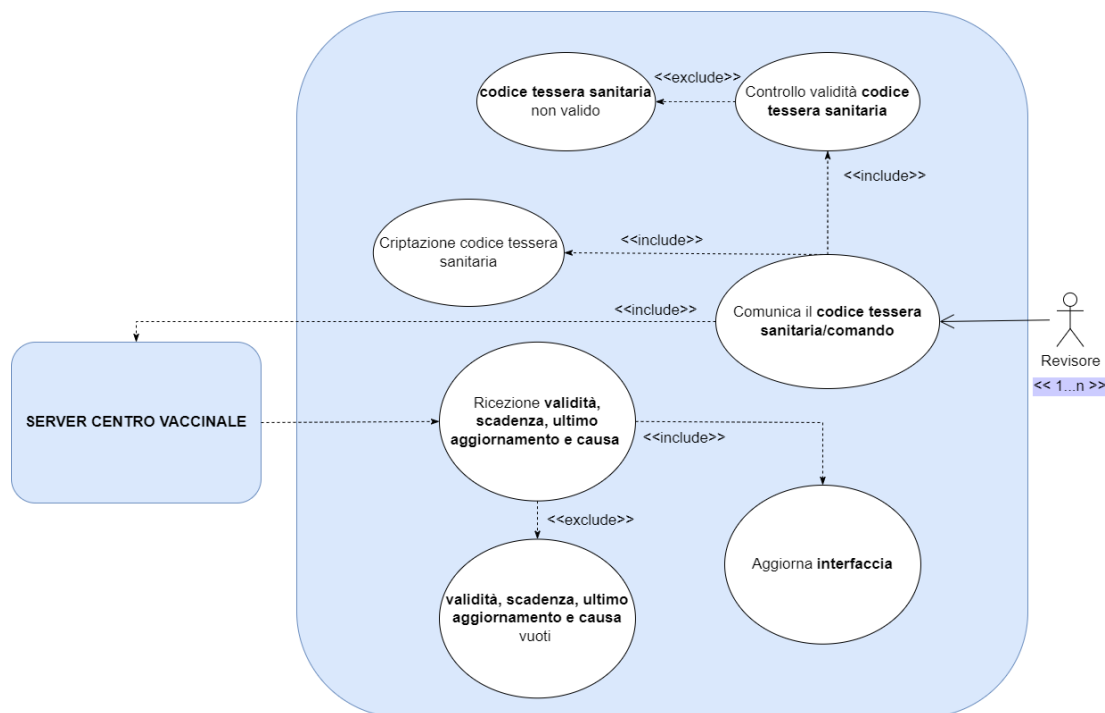
- Di seguito mostriamo il diagramma dei casi d'uso da cui si sono ricavate le informazioni elencate:



- **Client (revisore):** Il client è connesso solo con il server assistente e attraverso il corrispettivo menu potrà richiedere di visualizzare le informazioni relative al documento **Green Pass** ottenuto in fase di vaccinazione o guarigione, inserendo il proprio codice di tessera sanitaria (valido), con il quale si è iscritto alla piattaforma.
In particolare il client eseguirà le seguenti azioni:

1. **Richiesta informazioni relative al documento **Green Pass**:** Il client revisore, una volta inserito un codice di tessera sanitaria valido e già presente nella piattaforma, comunicherà con il server assistente per richiedere le informazioni relative alla validità, scadenza ed ultimo aggiornamento del documento **Green Pass**. In caso di errore, il client viene notificato relativamente la natura di quest'ultimo.

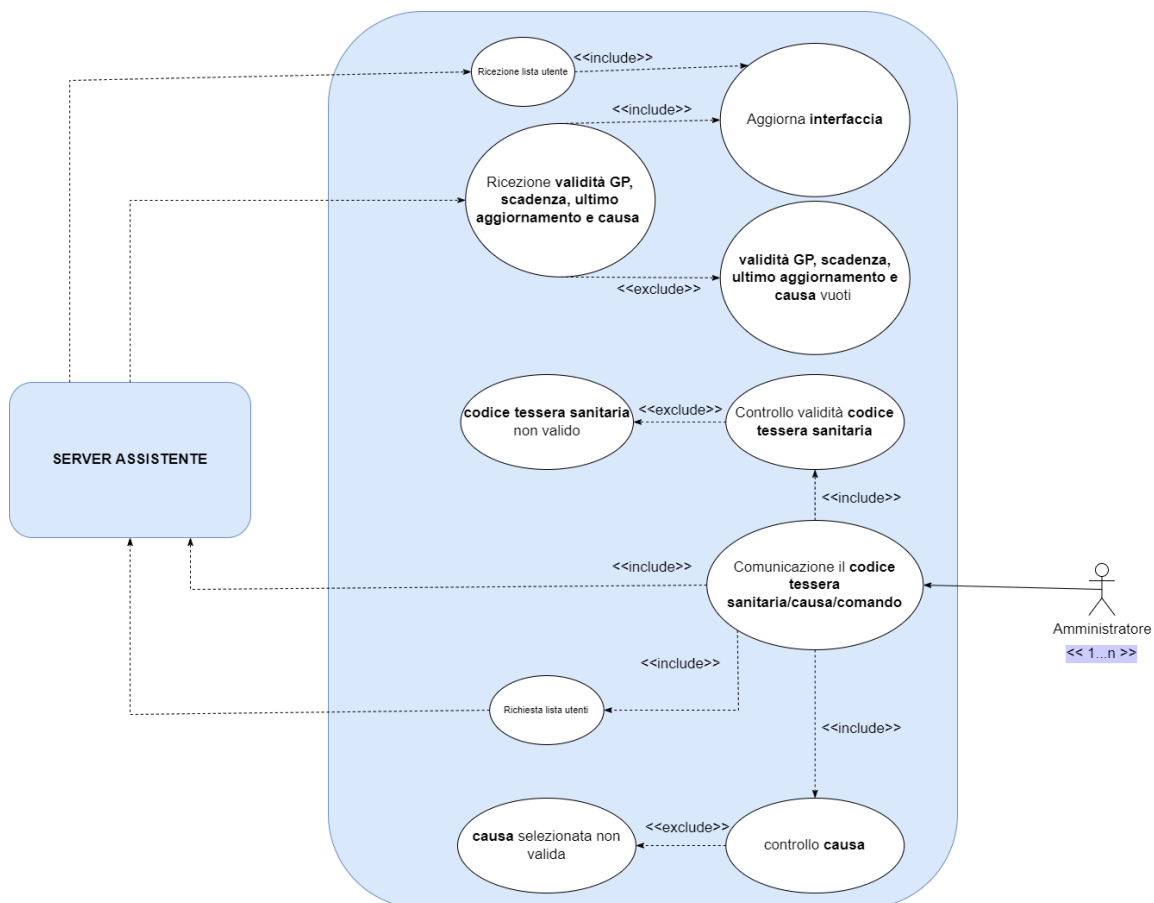
Di seguito mostriamo il diagramma dei casi d'uso da cui si sono ricavate le informazioni elencate:



- **Client (amministratore):** Il client è connesso solo con il server assistente e attraverso un menu, dove è possibile visualizzare la lista degli utenti iscritti alla piattaforma **Green Pass**, potrà selezionare uno degli utenti ed una delle possibili cause (covid o guarigione) per aggiornare le informazioni relative al corrispettivo **Green Pass**.
In particolare il client eseguirà le seguenti azioni:

1. **Visualizzare la lista degli utenti iscritti alla piattaforma **Green Pass**:** Il client amministratore, potrà selezionare solo utenti iscritti alla piattaforma **Green Pass**, pertanto, si rende necessario mostrare al client, una lista utenti. Tale azione viene effettuata senza richiedere alcuna interazione al client, inviando un'esplicita richiesta al server assistente.
2. **Aggiornare le informazioni relative al documento **Green Pass** di uno specifico utente:** il client amministratore, una volta selezionato un utente ed una causa dal corrispettivo menu, effettuerà una richiesta di aggiornamento delle informazioni **Green Pass** al server assistente.

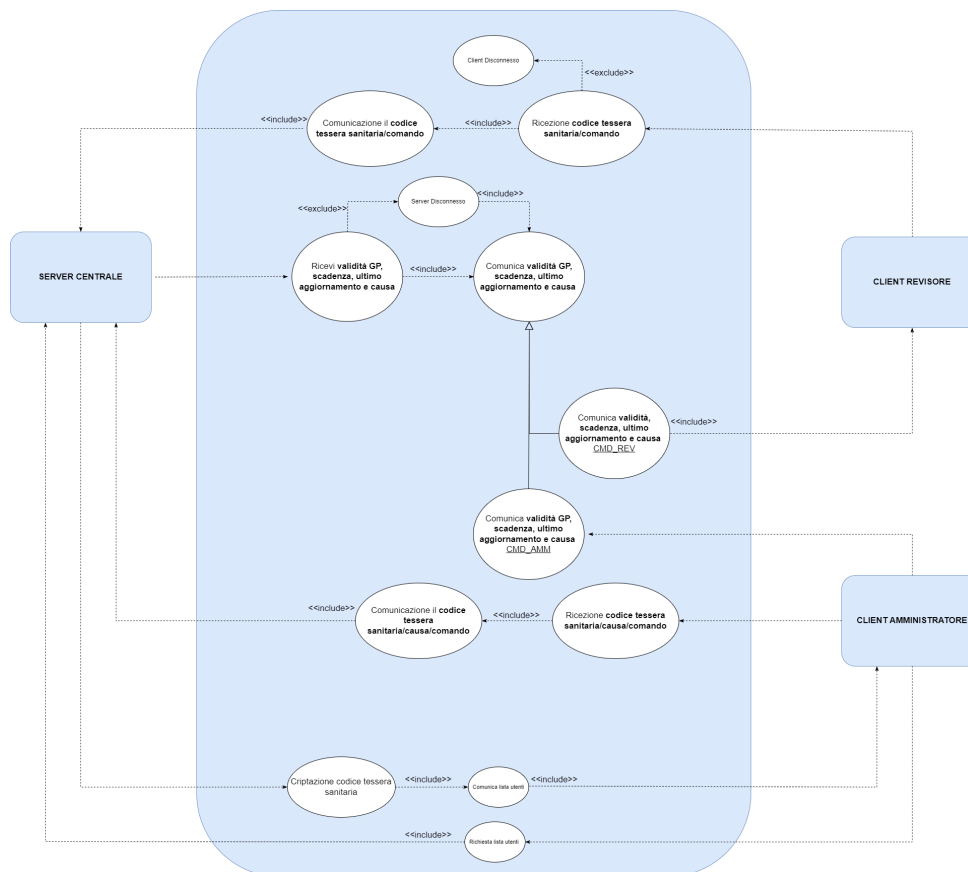
Di seguito mostriamo il diagramma dei casi d'uso da cui si sono ricavate le informazioni elencate:



- **Server (assistente):** Il seguente è connesso al server centrale e ai client “amministratore” e “revisore”. Tale entità si occuperà di ricevere le richieste da entrambi i client e di inviarle al server centrale, in modo da non sovraccaricare la coda delle connessioni di quest’ultimo. In particolare il server eseguirà le seguenti azioni:

1. **Richiedere la lista degli utenti iscritti alla piattaforma Green Pass:** il server assistente dopo aver ricevuto la richiesta dal client amministratore, del reperimento lista degli utenti iscritti alla piattaforma **Green Pass**, si occuperà di effettuare una nuova richiesta al server centrale, per ottenere quest’ultima. Infine, una volta ottenuta la lista dal server centrale, si occuperà di decriptarla ed inviarla al client amministratore.
2. **Richiedere di aggiornare le informazioni relative al documento Green Pass di uno specifico utente:** il server assistente dopo aver ricevuto la richiesta dal client amministratore di modifica delle informazioni relative ad uno specifico documento **Green Pass**, si occuperà di effettuare una nuova richiesta al server centrale, per ottenere in risposta le informazioni del documento **Green Pass** modificato. Infine, comunicherà tali modifiche al client amministratore.
3. **Richiesta informazioni relative al documento Green Pass:** il server assistente dopo aver ricevuto la richiesta dal client revisore di ottenimento delle informazioni relative ad uno specifico documento **Green Pass**, si occuperà di effettuare una nuova richiesta al server centrale per ottenere le informazioni necessarie e successivamente inviarle al client revisore.

Di seguito mostriamo il diagramma dei casi d’uso da cui si sono ricavate le informazioni elencate:



- **Server Centrale:** è il cuore della comunicazione tra i client e i vari server. Il seguente è connesso al server centro vaccinale e al server assistente. Tale entità si occuperà di ricevere le richieste smistate dai due server e di servirle fornendo diversi pacchetti in response in base alla richiesta arrivata. Inoltre, si occuperà di gestire il file contenente le informazioni relative agli utenti iscritti, in modo da garantire un accesso atomico alle informazioni.

In particolare il server eseguirà le seguenti azioni:

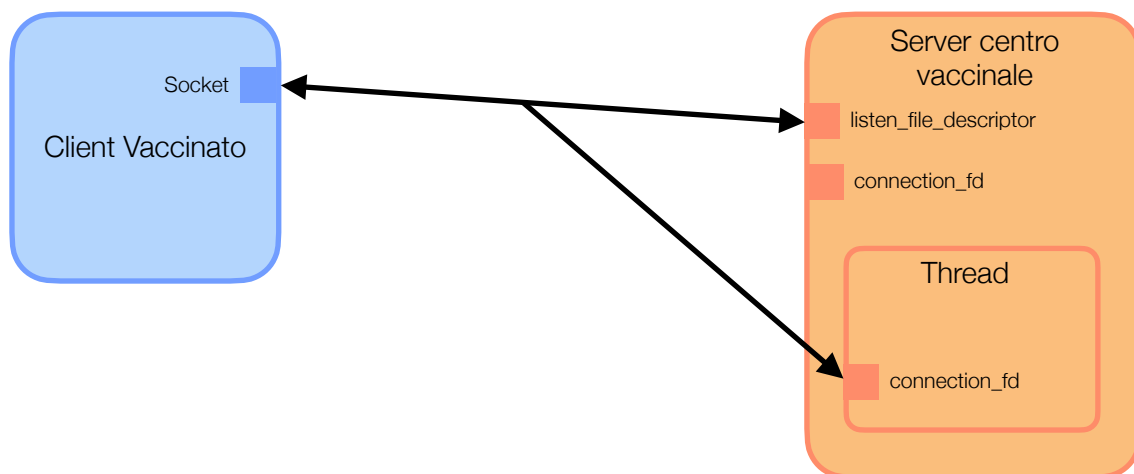
1. **Controllo utente già iscritto alla piattaforma:** il server centrale dopo aver ricevuto esplicita richiesta dal server centro vaccinale, di verificare se un utente risulta essere già iscritto alla piattaforma **Green Pass**, procede con la lettura del file contenente la lista degli utenti iscritti. Successivamente, ne verifica, l'eventuale avvenuta iscrizione, confrontando il codice tessera sanitaria ricevuto dal server centro vaccinale, con i codici presenti nel file. Infine, verificata l'iscrizione, fornirà in response al server centro vaccinale un flag che permetterà di interpretare l'esito dell'operazione. Tale azione, gestendo delle operazioni su file può riscontrare delle eccezioni, pertanto il server centrale si occuperà di allegare, insieme alla response da inviare, al server centro vaccinale, un pacchetto contenente dei flag di errore (apertura, lettura e scrittura) avvenuti durante l'esecuzione e la gestione della richiesta ricevuta. In questo modo il client, quando riceverà la response dal server centro vaccinale, potrà interpretare tali errori per aggiornare l'interfaccia in modo appropriato.
2. **Iscrizione alla piattaforma **Green Pass**:** il server centrale dopo aver ricevuto esplicita richiesta dal server centro vaccinale, di iscrizione alla piattaforma **Green Pass** di uno specifico utente, procederà scrivendo sul file, le informazioni relative all'utente, ricevute dal server centro vaccinale. Infine, effettuata l'iscrizione, fornirà in response al server centro vaccinale un flag che permetterà di interpretare l'esito dell'operazione. Tale azione, gestendo delle operazioni su file può riscontrare delle eccezioni, pertanto il server centrale si occuperà di allegare, insieme alla response da inviare, al server centro vaccinale, un pacchetto contenente dei flag di errore (apertura, lettura e scrittura) avvenuti durante l'esecuzione e la gestione della richiesta ricevuta. In questo modo il client, quando riceverà la response dal server centro vaccinale, potrà interpretare tali errori per aggiornare l'interfaccia in modo appropriato.

- [illegible]

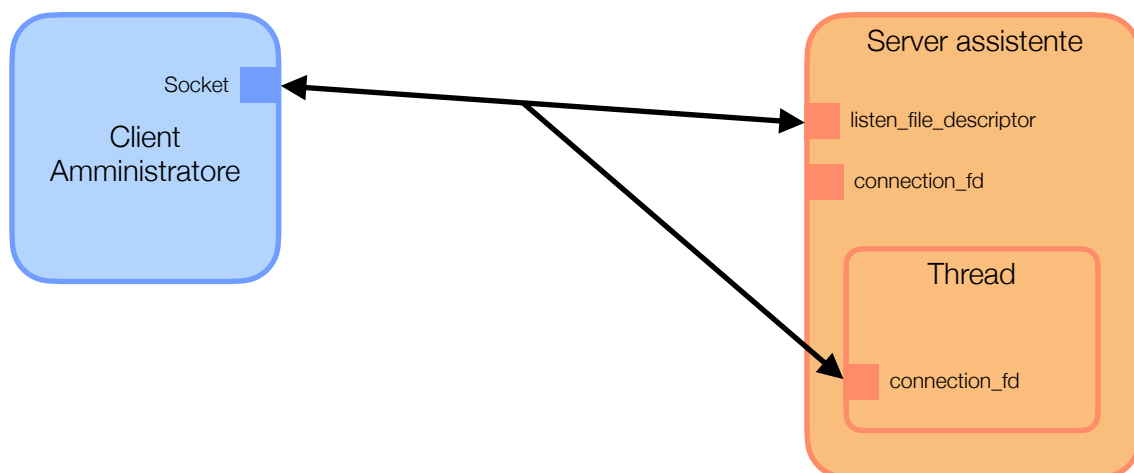
2.0 - Descrizione e schemi dell'architettura

La descrizione degli schemi dell'architettura permette di comprendere al meglio come avviene la comunicazione tra le diverse entità.

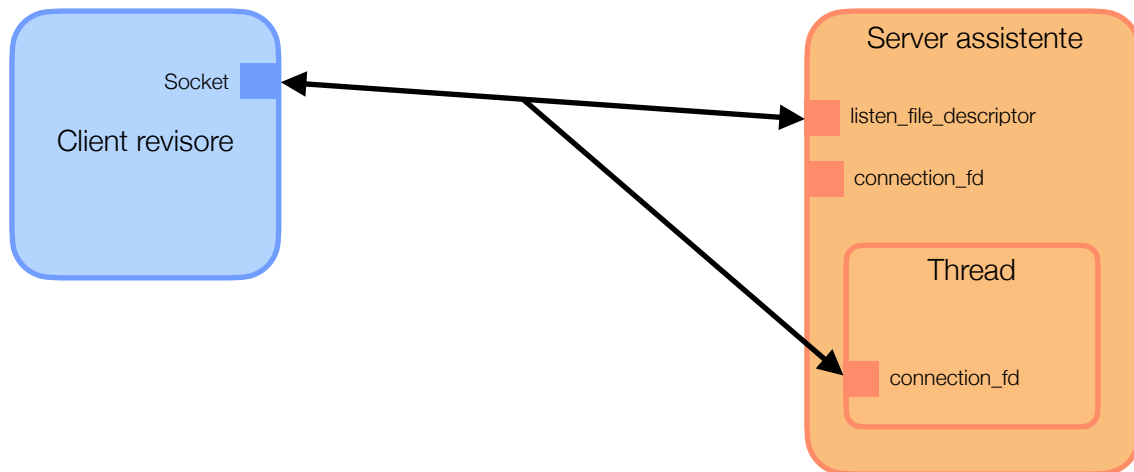
La prima comunicazione avviene tra il client vaccinato e il server centro vaccinale. Il client vaccinato si connette al server centro vaccinale tramite la creazione del socket, valorizza le strutture dati e il server si mette in modalità ascolto, in attesa che i client si connettano. Tutte le richieste verranno gestite concorrentemente dal server, attraverso la creazione di un thread, in quanto appena arriva una richiesta, il server genera un nuovo thread che si occuperà di comunicare con il client connesso, mentre il master thread si rimetterà in modalità ascolto per accettare altre richieste.



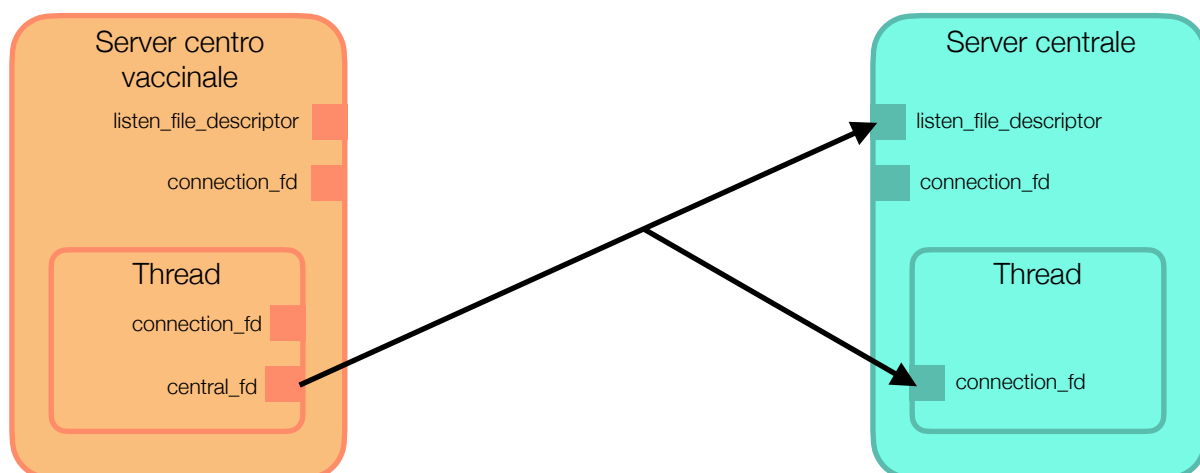
La seconda comunicazione avviene tra il client amministratore e il server assistente. Il client amministratore si connette al server assistente tramite la creazione del socket, valorizza le strutture dati e il server si mette in modalità ascolto, in attesa che i client si connettano. Tutte le richieste verranno gestite concorrentemente dal server, attraverso la creazione di un thread, in quanto appena arriva una richiesta, il server genera un nuovo thread che si occuperà di comunicare con il client connesso, mentre il master thread si rimetterà in modalità ascolto per accettare altre richieste.



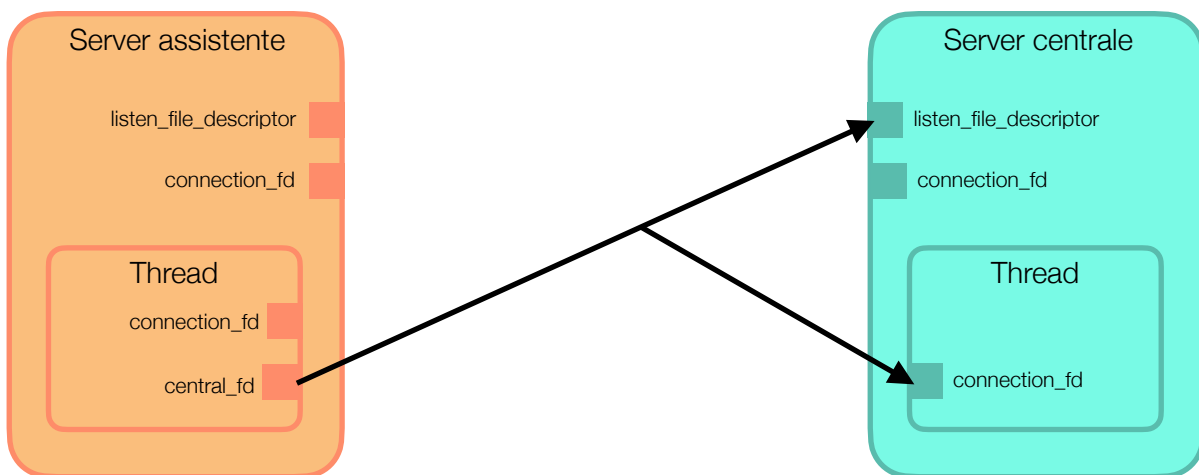
La terza comunicazione avviene tra il client revisore e il server assistente. Il client revisore si connette al server assistente tramite la creazione del socket, valorizza le strutture dati e il server si mette in modalità ascolto, in attesa che i client si connettano. Tutte le richieste verranno gestite concorrentemente dal server, attraverso la creazione di un thread, in quanto appena arriva una richiesta, il server genera un nuovo thread che si occuperà di comunicare con il client connesso, mentre il master thread si rimetterà in modalità ascolto per accettare altre richieste.



La quarta comunicazione avviene tra il server centro vaccinale e server centrale. Si può osservare che in questa particolare comunicazione il server centro vaccinale si comporta da client per il server centrale. In questo caso, il server centro vaccinale, ha già ricevuto una richiesta dal client vaccinato, gestita in un thread separato. In tale thread, il server centro vaccinale si connette al server centrale tramite la creazione di una nuova socket, valorizzando le strutture dati. Il server centrale si mette in modalità ascolto, in attesa che i client si connettano. Tutte le richieste verranno gestite concorrentemente dal server centrale, attraverso la creazione di un thread, in quanto appena arriva una richiesta, il server genererà un nuovo thread che si occuperà di comunicare con il client (server centro vaccinale) connesso, mentre il master thread si rimetterà in modalità ascolto per accettare altre richieste.



L'ultima comunicazione avviene tra il server assistente e server centrale. Si può osservare che in questa particolare comunicazione il server assistente si comporta da client per il server centrale. In questo caso, il server assistente, ha già ricevuto una richiesta dal client amministratore o dal client revisore, gestita in un thread separato. In tale thread, il server assistente si connette al server centrale tramite la creazione di una nuova socket, , valorizzando le strutture dati. Il server centrale si mette in modalità ascolto, in attesa che i client si connettano. Tutte le richieste verranno gestite concorrentemente dal server centrale, attraverso la creazione di un thread, in quanto appena arriva una richiesta, il server genera un nuovo thread che si occuperà di comunicare con il client (server assistente) connesso, mentre il master thread si rimetterà in modalità ascolto per accettare altre richieste.



3.0 - Descrizione e schemi del protocollo applicazione

In questa sezione verrà descritto in modo dettagliato il protocollo livello applicazione adottato nel progetto.

Nella prima fase abbiamo ideato una serie di regole utili al funzionamento del sistema **Green Pass**. Di seguito le elenchiamo descrivendole:

- **L'utente vaccinato deve inserire la data di vaccinazione oltre al codice di tessera sanitaria:** abbiamo ipotizzato che l'utente vaccinato possa iscriversi alla piattaforma **Green Pass** inserendo una data di vaccinazione che sia precedente o uguale alla data odierna. Pertanto, viene data la possibilità all'utente del client vaccinato di inserire in autonomia una data di vaccinazione valida;
- **La data di vaccinazione inserita dall'utente del Client vaccinato,** non viene considerata valida se inferiore a un mese dalla data odierna;
- **La data di vaccinazione inserita dall'utente del Client vaccinato, non può essere superiore rispetto la data odierna,** in quanto è impossibile prenotare documenti **Green Pass** in base alle vaccinazioni future;
- **La data di vaccinazione inserita dal client vaccinato** deve essere conforme al formato gg/mm/yyyy;
- **Il client vaccinato effettua una richiesta daytime locale al server centro vaccinale:** abbiamo previsto una richiesta daytime da parte del client vaccinato verso il server centro vaccinale per verificare che la data inserita rispetti le condizioni appena elencate. Inoltre, tale richiesta ha lo scopo di deresponsabilizzare il client dalla verifica della data, in quanto, quest'ultima potrebbe dipendere fortemente dalle impostazioni orarie del client;
- **L'utente del client vaccinato deve inserire un codice di tessera sanitaria valido:** l'utente non potrà eseguire alcuna registrazione se non inserirà un codice di tessera sanitaria validato dal client vaccinato. Un codice di tessera sanitaria viene ritenuto valido se strutturato nel seguente modo:

xxxxx-xxxxx-xxxxxxxxxxx

dove le prime 5 cifre, equivalenti a 80380, riguardano l'istituzione sanitaria specifica, pertanto, tale codice sarà uguale per tutti. Le successive 5 cifre indicano il codice della regione, contenuto in un file separato. Le restanti 10 cifre possono essere valori numerici casuali
- **L'utente del client vaccinato dovrà accettare l'informativa sul trattamento dei dati:** abbiamo ipotizzato che un utente non possa iscriversi senza accettare l'informativa sul trattamento dei dati. Nel caso in cui tale condizione non sia accettata, il sistema non invierà la richiesta di iscrizione al server centro vaccinale;
- **Il client vaccinato dovrà verificare la validità della data di vaccinazione e del codice di tessera sanitaria prima di inviare una richiesta al server centro vaccinale:** se la data o il codice di tessera sanitaria inseriti dall'utente nel client vaccinato non sono validi, non vi dovrà essere alcuna comunicazione tra client vaccinato e server centro vaccinale;
- **Il client vaccinato prima di inviare una richiesta di iscrizione, si occuperà di criptare il proprio codice di tessera sanitaria,** in questo modo è possibile proteggere i dati, in quanto nessun codice di tessera sanitaria sarà associabile ai dati **Green Pass** generati durante il processo di vaccinazione, guarigione o contrazione del covid.

- **Un utente può vaccinarsi solo una volta:** abbiamo ipotizzato che un utente possa conseguire una sola vaccinazione, pertanto, se un soggetto tenta di comunicare al server centro vaccinale

per la seconda volta il proprio codice di tessera sanitaria, il client restituirà un errore ricevuto come risposta dal centro vaccinale.

- **Il server centro vaccinale controllerà se per il codice di tessera sanitaria ricevuto è già avvenuta un'iscrizione:** il server centro vaccinale, prima di calcolare la data di scadenza richiede al server centrale di verificare se l'utente sia già iscritto. Se l'iscrizione è avvenuta precedentemente, il server centro vaccinale invierà al client vaccinato un flag utile ad interpretare l'esito di tale procedura;
- **Il server centro vaccinale calcolerà la data di scadenza Green Pass** a partire dalla data di vaccinazione ricevuta dal client vaccinato;
- **Il documento Green Pass** avrà una durata di 6 mesi;
- **Il server centrale si occuperà di salvare tutte le informazioni all'interno di un file**, secondo la seguente struttura:

Codice tessera	Data scadenza Green Pass	Motivazione ultimo aggiornamento	Data ultimo aggiornamento
----------------	--------------------------	----------------------------------	---------------------------

Dove ogni campo viene separato da un carattere spazio.

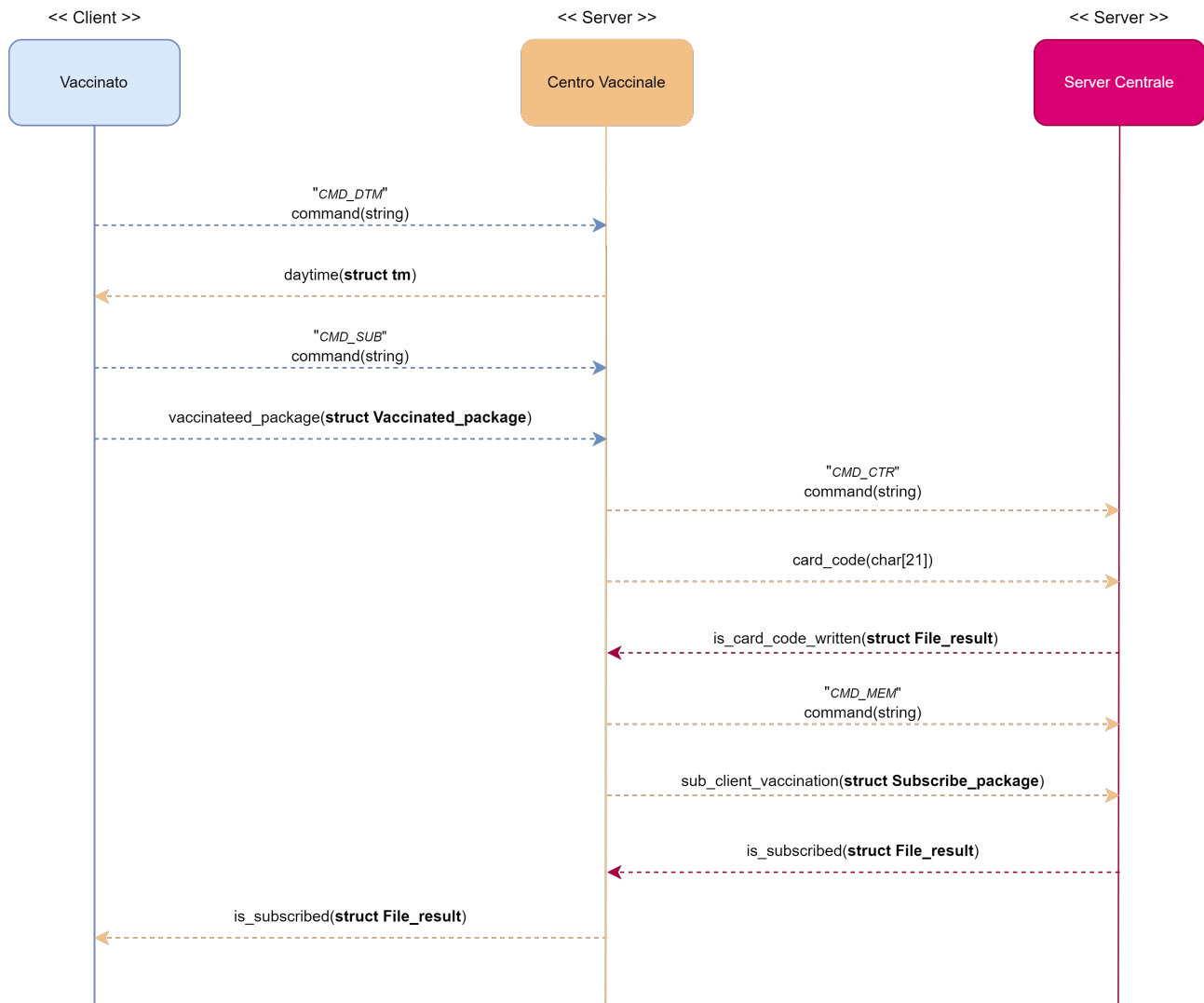
- **Il campo data ultimo aggiornamento verrà inizializzato con:**
 - la data di vaccinazione, nel caso in cui l'utente si stia iscrivendo alla piattaforma;
 - la data locale del server nel caso in cui l'amministratore effettui una modifica per covid o guarigione;
- **Il server centrale è l'unica entità che ha il permesso** di leggere e scrivere su file;
- **L'utente del client revisore deve inserire un codice di tessera sanitaria valido** per ricevere le informazioni desiderate relative al documento **Green Pass**;
- **L'utente del client revisore deve inserire un codice** di tessera sanitaria esistente nella piattaforma **Green Pass**;
- **Il client revisore prima di inviare una richiesta di revisione, si occuperà di criptare il codice** di tessera sanitaria associato ad uno specifico **Green Pass**, in questo modo, quando il codice di tessera sanitaria giungerà al server centrale, quest'ultimo lo potrà confrontare direttamente con i codici di tessera sanitaria già presenti nel file.
- **Il client amministratore deve scaricare la lista di codici** di tessera sanitaria degli utenti attualmente iscritti;
- **L'utente del client amministratore deve selezionare uno dei codici** presenti nella lista ricevuta dal server assistente.
- **L'utente del client amministratore non può modificare** le informazioni di un utente con causa guarigione se quest'ultimo non ha contratto il covid;
- **L'utente del client amministratore può modificare** le informazioni di un utente solo se la causa selezionata è diversa da quella dell'ultimo aggiornamento;
- **L'utente del client amministratore riceverà, in risposta alle modifiche**, le informazioni aggiornate relative al documento **Green Pass** modificato;
- **Il server assistente una volta ottenuta la lista dei codici di tessera sanitaria dal server centrale**, si dovrà occupare di decriptarli prima di inviarli al client amministratore.

Nella seconda fase abbiamo ideato dei diagrammi delle sequenze, in base alle regole precedentemente elencate.

Di seguito mostriamo le schematizzazioni per ogni richiesta.

3.1.0 - Client Vaccinato

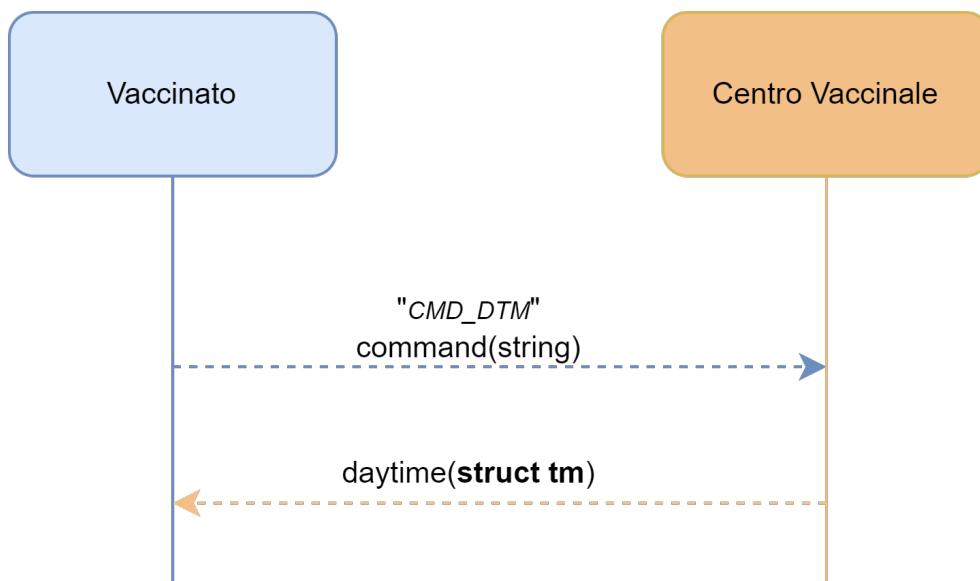
Per effettuare l'operazione di iscrizione di un utente attraverso il client vaccinato, sono richieste le seguenti azioni:



Analizziamo, il diagramma delle sequenze suddividendolo per richieste.

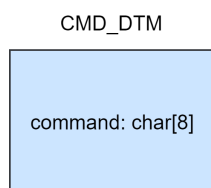
3.1.1 - Richiesta daytime

Inizialmente, dopo aver effettuato le operazioni di base per la creazione di un socket client e server, il client vaccinato si connette al server centro vaccinale, effettuando una *three way handshake*. Successivamente alla connessione, il client vaccinato invierà una richiesta daytime al server centro vaccinale, secondo le seguenti specifiche:



Prima comunicazione

Il server centro vaccinale può gestire molteplici richieste, pertanto, per effettuare una qualsiasi operazione, sarà necessario inviare un comando per eseguire un set di istruzioni predefinite. In questa prima comunicazione, dal client vaccinato al server centro vaccinale, viene inviato un comando: *CMD_DTM*, per identificare la richiesta di un daytime nel server centro vaccinale. Di seguito mostriamo una schematizzazione dello stream di dati inviato:



Seconda comunicazione

Nella seconda comunicazione, dal server centro vaccinale al client vaccinato, viene inviato un pacchetto del tipo struct tm, contenente le informazioni necessarie per rappresentare una data. Viene utilizzato tale tipo di dato, in quanto sarà necessario al client per ottimizzare l'esecuzione dell'operazione di confronto tra date. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

Struct tm

tm_sec:int	tm_min:int	tm_hour:int
tm_mday:int	tm_mon:int	tm_year:int
tm_wday:int	tm_yday:int	tm_isdst:int

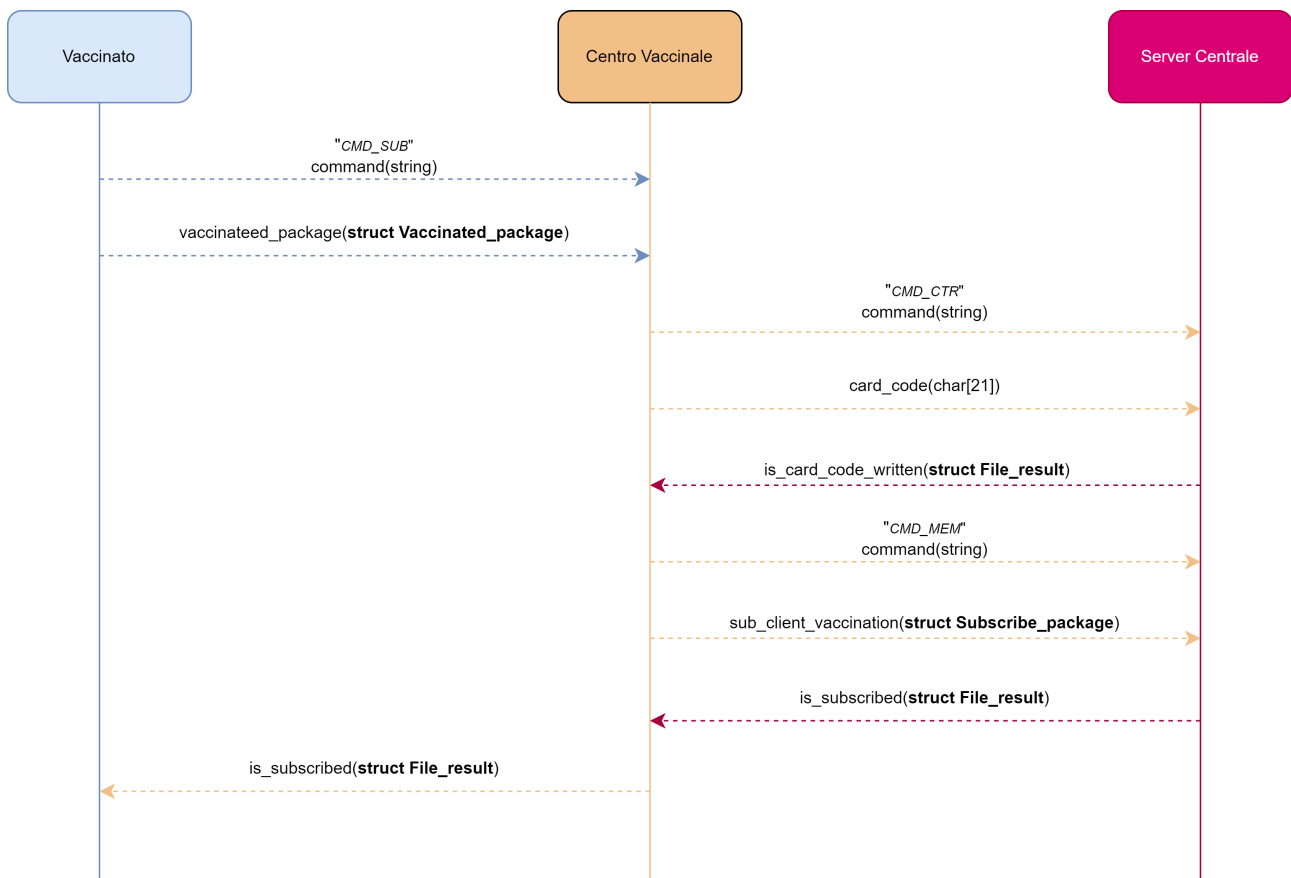
in particolare:

- **tm_sec**: secondi compresi in un range <0..60> dell'orario generato;
- **tm_min**: minuti compresi in un range <0..59> dell'orario generato;
- **tm_hour**: ore comprese in un range <0..23> dell'orario generato;

- **tm_mday**: giorno del mese compreso nel range <1..31> della data generata;
- **tm_mon**: mese dell'anno compreso nel range <0..11> della data generata;
- **tm_year**: anni passati dal 1900 relativi alla data generata;
- **tm_wday**: giorno della settimana nel range <0..6> della data generata;
- **tm_yday**: giorno dell'anno nel range <0..365> della data generata;
- **tm_isdst**: flag per definire l'ora legale.

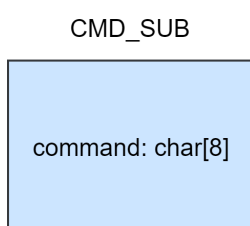
3.1.2 - Richiesta d'iscrizione con successo

La seguente richiesta viene eseguita temporalmente dopo la richiesta daytime, pertanto, le azioni preliminari come la creazione dei socket e la connessione tra client vaccinato e server centro vaccinale, sono già avvenute. Successivamente, il client vaccinato invierà una richiesta d'iscrizione al server centro vaccinale, secondo le seguenti specifiche:



Prima comunicazione

Il server centro vaccinale può gestire molteplici richieste, pertanto, per effettuare una qualsiasi operazione, sarà necessario inviare un comando per eseguire un set di istruzioni predefinite. In questa prima comunicazione, dal client vaccinato al server centro vaccinale, viene inviato un comando: *CMD_SUB*, per identificare la richiesta d'iscrizione nel server centro vaccinale. Di seguito mostriamo una schematizzazione dello stream di dati inviato:



Seconda comunicazione

Nella seconda comunicazione, dal client vaccinato al server centro vaccinale, viene inviato un pacchetto del tipo vaccinated package, contenente le informazioni relative al codice di tessera sanitaria e alla data di vaccinazione dell'utente. Tali informazioni sono necessarie ai fini dell'iscrizione alla piattaforma **Green Pass**. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

Vaccinated_package

codice_tessera_sanitaria: char[21]
data_di_vaccinazione: Struct tm

in particolare:

- **codice tessera sanitaria**: campo utile a rappresentare il pacchetto da comunicare al server centro vaccinale, per procedere alla sottoscrizione del sistema **Green Pass**;
- **data di vaccinazione**: data di vaccinazione dell'utente, memorizzata attraverso una struct tm.

Terza comunicazione

La terza comunicazione, avviene tra server centro vaccinale e server centrale, in questa comunicazione il server centro vaccinale si comporta da client per il server centrale. Quindi, dopo aver svolto le azioni preliminari, il server centro vaccinale si connette al server centrale ed invia una richiesta di controllo, attraverso il comando: *CMD_CTR*, utile a determinare se un utente è già iscritto. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

CMD_CTR

command: char[8]

Quarta comunicazione

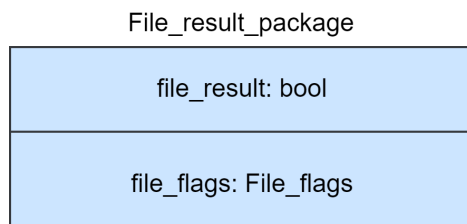
Nella quarta comunicazione, dal server centro vaccinale al server centrale, viene inviato il codice di tessera sanitaria ricevuto dal client vaccinato, di cui il server centrale deve verificare l'iscrizione. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

card_code: char[21]

Quinta comunicazione

Nella quinta comunicazione, dal server centrale al server centro vaccinale, viene inviato un pacchetto contenente dei flag, che avranno lo scopo di essere interpretati dal server centro vaccinale per determinare se un utente è già iscritto alla piattaforma, oppure se vi sono stati errori

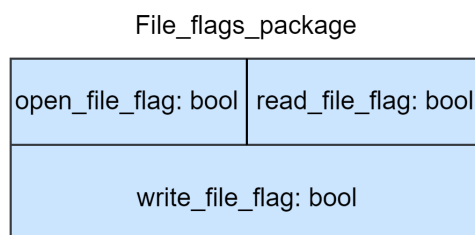
durante le operazioni di lettura o apertura dei file. Di seguito mostriamo una schematizzazione dello stream di dati inviato:



in particolare:

- **file result:** campo utile a indicare se un utente risulta essere già iscritto alla piattaforma;
- **file flags:** struttura utile a rappresentare gli errori che possono intercorrere durante diverse operazioni su file. In questo modo un client può essere notificato di eventuali errori che intercorrono sul server ed aggiornare l'interfaccia in base a tali condizioni.

Dove file_flags può essere schematizzato come segue:

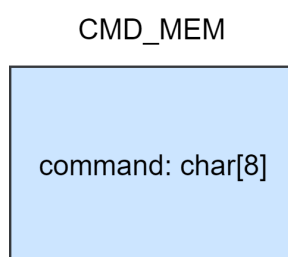


In particolare:

- **open file flag:** Valore booleano utile a verificare la presenza di errore durante l'operazione di apertura file
- **write file flag:** - Valore booleano utile a verificare la presenza di errore durante l'operazione di scrittura file
- **read file flag:** - Valore booleano utile a verificare la presenza di errore durante l'operazione di lettura file

Sesta comunicazione

Nella sesta comunicazione, dal server centro vaccinale al server centrale, dopo aver verificato che l'utente non sia già iscritto alla piattaforma, viene inviata una richiesta d'iscrizione al server centrale, attraverso il comando: *CMD_MEM*. Di seguito mostriamo una schematizzazione dello stream di dati inviato:



Settima comunicazione

Nella settima comunicazione, dal server centro vaccinale al server centrale, viene inviato un pacchetto contenente informazioni relative alla vaccinazione dell'utente, per finalizzare l'operazione d'iscrizione. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

Subscribe_package	
expiration_date: Struct tm	
vaccinated_package: struct Vaccinated_package	

In particolare:

- **expiration_date**: data di scadenza del documento **Green Pass**;
- **vaccinated_package**: informazioni relative alla data di vaccinazione e al codice di tessera sanitaria dell'utente (spiegata nel dettaglio nella seconda comunicazione);

Ottava comunicazione

Nell'ottava comunicazione, dal server centrale al server centro vaccinale, viene inviato un pacchetto contenente dei flag, che avranno lo scopo di essere interpretati dal server centro vaccinale per determinare se l'operazione d'iscrizione è andata a buon fine, oppure se vi sono stati errori durante le operazioni di scrittura o apertura dei file. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

File_result_package	
file_result: bool	
file_flags: File_flags	

in particolare:

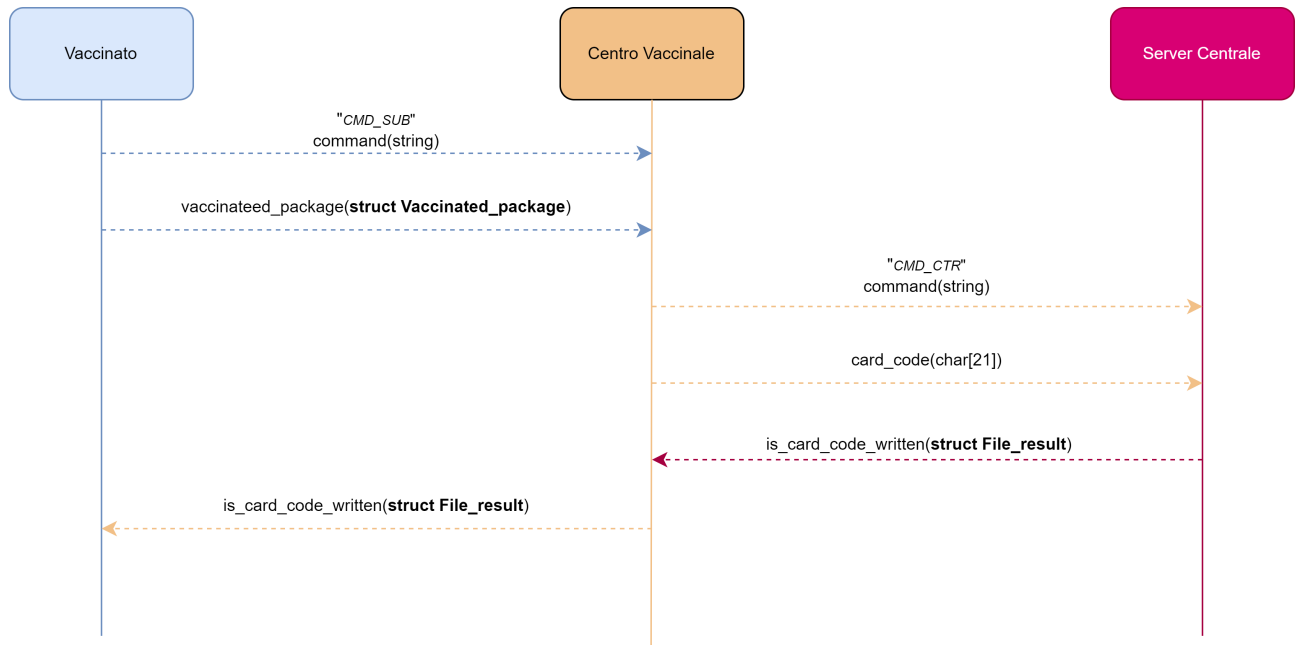
- **file_result**: campo utile a indicare se l'operazione d'iscrizione è andata a buon fine;
- **file_flags**: struttura utile a rappresentare gli errori che possono intercorrere durante diverse operazioni su file. In questo modo un client può essere notificato di eventuali errori che intercorrono sul server ed aggiornare l'interfaccia in base a tali condizioni (spiegata nel dettaglio nella quinta comunicazione).

Nona comunicazione

Nella nona comunicazione, dal server centro vaccinale al client vaccinato, verrà inviato a quest'ultimo, lo **stesso pacchetto** ricevuto dal server centrale nell'ottava comunicazione. In questo modo il client potrà aggiornare l'interfaccia in base ai dati ricevuti.

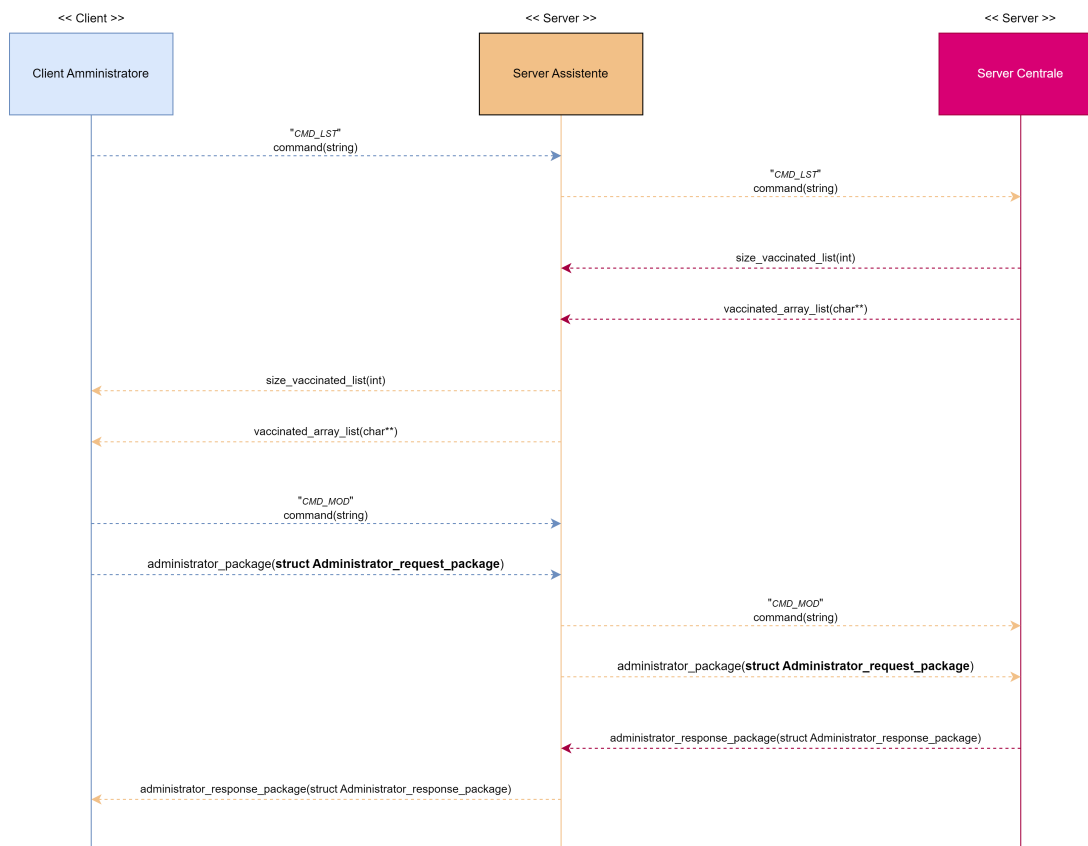
3.1.3 - Richiesta d'iscrizione senza successo

La seguente richiesta, eseguita temporalmente dopo la richiesta daytime, è del tutto analoga all'iscrizione con successo ([capitolo 3.1.2](#)), ma differisce da quest'ultima, in quanto consegna prematuramente, un pacchetto file result al client vaccinato, dato che, il server centrale ha restituito un valore positivo relativamente alla già avvenuta iscrizione alla piattaforma.



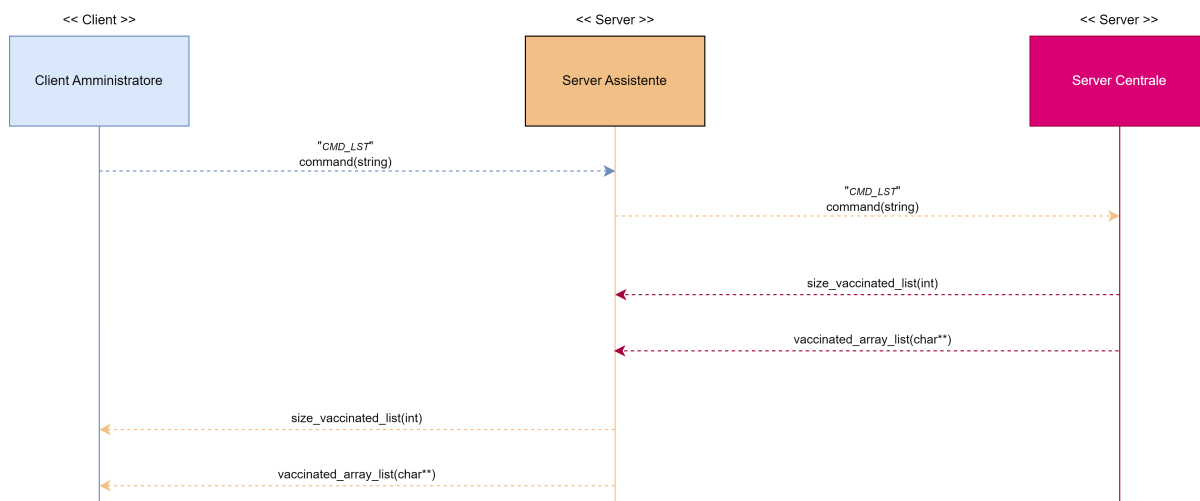
3.2.0 - Client Amministratore

Per effettuare l'operazione di modifica, relativamente alle informazioni del documento **Green Pass** di un utente, attraverso il client amministratore, sono richieste le seguenti azioni:



3.2.1 - Richiesta lista codici di tessera sanitaria

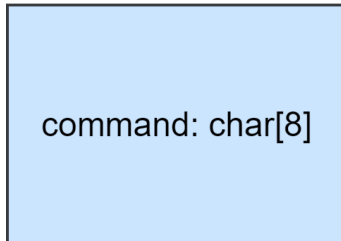
Inizialmente, dopo aver effettuato le operazioni di base per la creazione di un socket client e server, il client amministratore si connette al server assistente, effettuando una *three way handshake*. Successivamente alla connessione, il client amministratore invierà una richiesta al server assistente per ottenere la lista degli utenti che possono essere modificati.



Prima comunicazione

Il server assistente può gestire molteplici richieste, pertanto, per effettuare una qualsiasi operazione, sarà necessario inviare un comando per eseguire un set di istruzioni predefinite. In questa prima comunicazione, dal client amministratore al server assistente, viene inviato un comando: *CMD_LST*, per identificare la richiesta di ottenimento della lista utenti nel server assistente. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

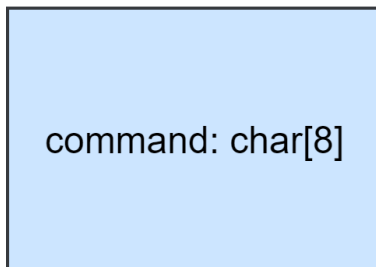
CMD_LST



Seconda comunicazione

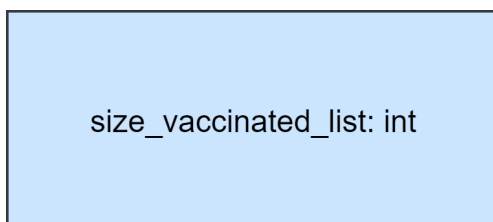
Il server centrale come il server assistente può gestire molteplici richieste, pertanto, per effettuare una qualsiasi operazione, sarà necessario inviare un comando per eseguire un set di istruzioni predefinite. Dopo la prima comunicazione il server assistente invierà lo stesso comando ricevuto dal client amministratore, al server centrale. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

CMD_LST



Terza comunicazione

Nella terza comunicazione, dal server centrale al server assistente, viene inviata una variabile di tipo intero, relativa alla dimensione della lista degli utenti, utile ad allineare i due server per il trasferimento di quest'ultima. Di seguito mostriamo una schematizzazione dello stream di dati inviato:



Quarta comunicazione

Nella quarta comunicazione, dal server centrale al server assistente, viene inviata la lista degli utenti iscritti alla piattaforma. Poiché, tale lista può avere dimensione variabile, i due server saranno allineati grazie alla terza comunicazione effettuata. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

```
vaccinated_array_list: char**
```

Quinta comunicazione

Nella quinta comunicazione, dal server assistente al client amministratore, verrà inviata a quest'ultimo, la **stessa variabile** ricevuta dal server centrale nella terza comunicazione. In questo modo il client ed il server saranno allineati per il trasferimento della lista. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

```
size_vaccinated_list: int
```

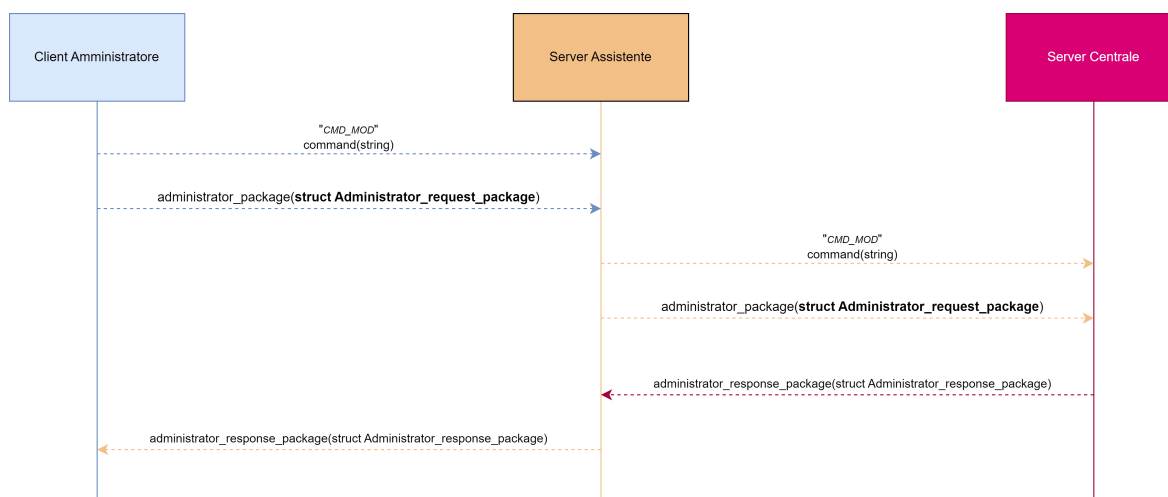
Sesta comunicazione

Nella sesta comunicazione, dal server assistente al client amministratore, verrà inviata a quest'ultimo, la **stessa variabile** ricevuta dal server centrale nella quarta comunicazione. In questo modo il client riceverà la lista degli utenti richiesti nella prima comunicazione.

```
vaccinated_array_list: char**
```

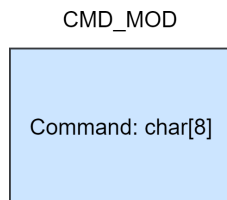
3.2.2 - Richiesta di modifica utente

La seguente richiesta viene eseguita temporalmente dopo l'ottenimento della lista codici di tessera sanitaria, pertanto, le azioni preliminari come la creazione dei socket e la connessione tra client amministratore e server assistente, sono già avvenute. Successivamente, il client amministratore invierà una richiesta di modifica informazioni utente al server assistente, secondo le seguenti specifiche:



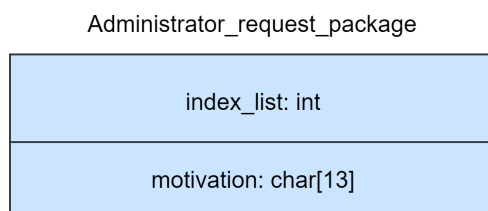
Prima comunicazione

Il server assistente può gestire molteplici richieste, pertanto, per effettuare una qualsiasi operazione, sarà necessario inviare un comando per eseguire un set di istruzioni predefinite. In questa prima comunicazione, dal client amministratore al server assistente, viene inviato un comando: *CMD_MOD*, per identificare la richiesta di modifica delle informazioni di un utente nel server assistente. Di seguito mostriamo una schematizzazione dello stream di dati inviato:



Seconda comunicazione

Nella seconda comunicazione, dal client amministratore al server assistente, viene inviato un pacchetto contenente informazioni relative alla modifica da effettuare. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

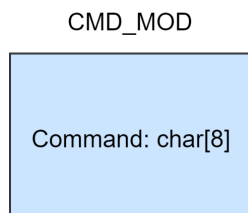


In particolare:

- **index list**: indice di riga corrispondente al codice di tessera sanitaria presente nel file;
- **motivation**: motivazione per la modifica che si vuole applicare (covid o guarigione).

Terza comunicazione

Il server centrale come il server assistente può gestire molteplici richieste, pertanto, per effettuare una qualsiasi operazione, sarà necessario inviare un comando, per eseguire un set di istruzioni predefinite. Dopo la seconda comunicazione il server assistente invierà lo stesso comando ricevuto durante la prima comunicazione dal client amministratore, al server centrale. Di seguito mostriamo una schematizzazione dello stream di dati inviato:



Quarta comunicazione

Nella quarta comunicazione, dal server assistente al server centrale, verrà inviato a quest'ultimo lo **stesso pacchetto** ricevuto dal client amministratore nella seconda comunicazione. In questo modo il client ed il server saranno allineati per il trasferimento della lista.

Di seguito mostriamo una schematizzazione dello stream di dati inviato:

Administrator_request_package

index_list: int
motivation: char[13]

Quinta comunicazione

Nella quinta comunicazione, dal server centrale al server amministratore, verrà inviato a quest'ultimo, un pacchetto di risposta contenente le informazioni relative alla modifica del documento **Green Pass** dell'utente specificato durante la richiesta del client. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

Administrator_response_package

not_updatable: bool	code: char[21]
reviser_package: Reviser_package	

in particolare:

- **not_updatable**: flag utile ad identificare se le informazioni utente sono modificabili;
- **code**: codice di tessera sanitaria dell'utente modificato;
- **reviser_package**: informazioni relative alla modifica del documento **Green Pass**.

Dove reviser_package può essere schematizzato come segue:

Reviser_package

is_green_pass_valid: bool	expiration_date: Struct tm
last_update: Struct tm	motivation: char[13]
file_flags: File_flags	

In particolare:

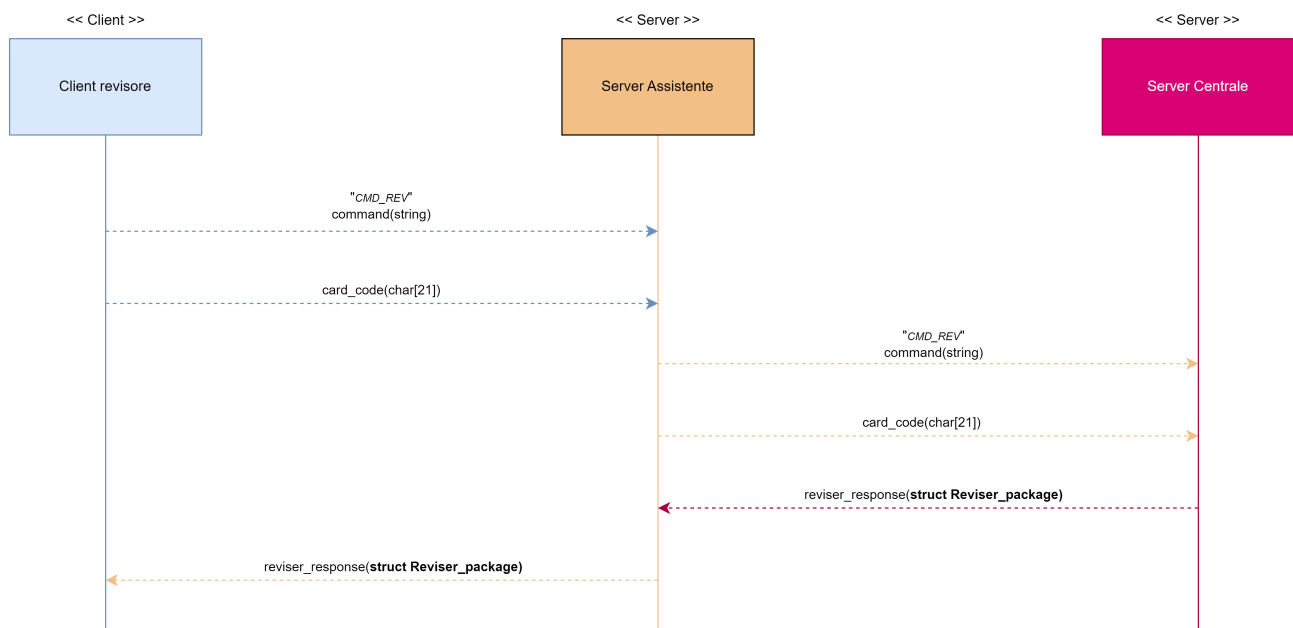
- **is green pass valid**: variabile che identifica se il **Green Pass** di un utente è valido o meno;
- **expiration date**: data di scadenza del **Green Pass**;
- **last update**: data dell'ultimo aggiornamento di stato del documento **Green Pass**;
- **motivation**: motivazione (vaccinazione, covid o guarigione) dell'ultimo aggiornamento di stato del documento **Green Pass**;
- **file flags**: struttura utile a rappresentare gli errori che possono intercorrere durante diverse operazioni su file. In questo modo un client può essere notificato di eventuali errori che intercorrono sul server ed aggiornare l'interfaccia in base a tali condizioni.

Sesta comunicazione

Nella sesta comunicazione, dal server assistente al client amministratore, verrà inviato a quest'ultimo, lo **stesso pacchetto** ricevuto dal server centrale nella quinta comunicazione. In questo modo il client potrà aggiornare l'interfaccia in base ai dati ricevuti.

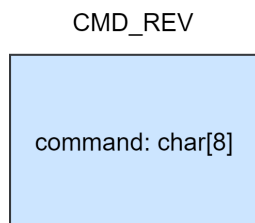
3.3.0 - Client Revisore

Per effettuare l'operazione di revisione delle informazioni **Green Pass** di uno specifico utente, attraverso il client revisore, sono richieste le seguenti azioni:



Prima comunicazione

Il server assistente può gestire molteplici richieste, pertanto, per effettuare una qualsiasi operazione, sarà necessario inviare un comando per eseguire un set di istruzioni predefinite. In questa prima comunicazione, dal client revisore al server assistente, viene inviato un comando: **CMD_REV**, per identificare la richiesta di ottenimento delle informazioni del documento **Green Pass** nel server assistente. Di seguito mostriamo una schematizzazione dello stream di dati inviato:



Seconda comunicazione

Nella seconda comunicazione, dal client revisore al server assistente, viene inviata una variabile relativa al codice di tessera sanitaria. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

card_code: char[21]

Terza comunicazione

Il server centrale come il server assistente può gestire molteplici richieste, pertanto, per effettuare una qualsiasi operazione, sarà necessario inviare un comando per eseguire un set di istruzioni predefinite. Dopo la seconda comunicazione il server assistente invierà lo stesso comando ricevuto durante la prima comunicazione dal client revisore, al server centrale. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

CMD_REV

command: char[8]

Quarta comunicazione

Nella quarta comunicazione, dal server assistente al server centrale, verrà inviato a quest'ultimo la **stessa variabile** ricevuta dal client revisore nella seconda comunicazione. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

card_code: char[21]

Quinta comunicazione

Nella quinta comunicazione, dal server centrale al server amministratore, verrà inviato a quest'ultimo, un pacchetto di risposta contenente le informazioni relative al documento **Green Pass** di un utente specificato durante la richiesta del client. Di seguito mostriamo una schematizzazione dello stream di dati inviato:

Reviser_package

is_green_pass_valid:bool	expiration_date: Struct tm
last_update: Struct tm	motivation: char[13]
file_flags: File_flags	

In particolare:

- **is green pass valid**: variabile che identifica se il **Green Pass** di un utente è valido o meno;
- **expiration date**: data di scadenza del **Green Pass**;
- **last update**: data dell'ultimo aggiornamento di stato del documento **Green Pass**;
- **motivation**: motivazione (vaccinazione, covid, guarigione) dell'ultimo aggiornamento di stato del documento **Green Pass**;
- **file flags**: struttura utile a rappresentare gli errori che possono intercorrere durante diverse operazioni su file. In questo modo un client può essere notificato di eventuali errori che intercorrono sul serve ed aggiornare l'interfaccia in base a tali condizioni.

Sesta comunicazione

Nella sesta comunicazione, dal server assistente al client revisore, verrà inviato a quest'ultimo lo **stesso pacchetto** ricevuto dal server centrale nella quinta comunicazione. In questo modo il client potrà aggiornare l'interfaccia in base ai dati ricevuti.

4.0 - Dettagli implementativi dei client

In questa sezione andremo ad analizzare i dettagli implementativi dei client, per capire come si è deciso di strutturarli.

4.1 - Client Vaccinato

Per il client vaccinato si è deciso di implementare un DNS diretto, in modo che possa accettare anche stringhe, vicine al linguaggio umano, in versione diversa dalla dotted decimal. Di seguito ne mostriamo lo snippet di codice:

```
if((server_dns = gethostbyname(IP_ADDRESS)) == NULL)
{
    /* La seguente funzione produce un messaggio sullo standard error (file descriptor: 2)
    che descrive la natura dell'errore */
    perror("DNS error: ");

    /* Interrompiamo l'esecuzione del programma con un errore */
    exit(EXIT_FAILURE);
}
```

Successivamente viene inizializzato il valore del file descriptor, sfruttando la funzione wrapper definita nella libreria socket utility. In questo modo associamo il file descriptor ad una socket:

```
client_file_descriptor = SocketIPV4();
```

Valorizziamo la struttura dati utile a costruire un endpoint da collegare:

```
server_address.sin_family = server_dns->h_addrtype;
server_address.sin_addr = *((struct in_addr *)server_dns->h_addr_list[0]);
server_address.sin_port = htons(SERVER_PORT);
```


Eseguiamo una three way handshake con la struttura precedentemente inizializzata, attraverso la funzione wrapper definita nella libreria `socket_utility`:

```
ConnectIPV4(client_file_descriptor, &server_address);
```

Inviando una richiesta `CMD_DTM` per ottenere un daytime dal server da confrontare con la data di vaccinazione inserita dall'utente:

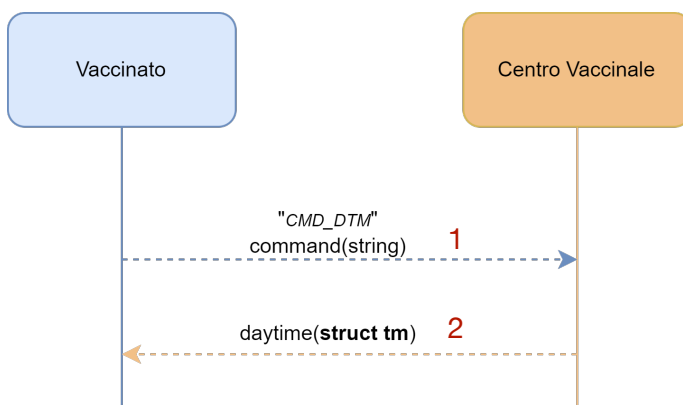
```
FullWrite(client_file_descriptor, command_writer_buffer, CMD_BUFFER_LEN); 1
if(FullRead(client_file_descriptor, server_daytime, sizeof(*server_daytime)) < 0) 2
{
    /* Caso in cui il server si sia disconnesso */

    fprintf(stderr, "Server disconnesso\n");

    /* Liberiamo la memoria precedentemente allocata dinamicamente nella memoria
    heap tramite una "@malloc" */
    free(server_daytime);
    free(client_daytime);
    free(verification_code);

    /* Chiusura del socket file descriptor connesso al server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_FAILURE);
}
```

Così come è indicato dal diagramma delle sequenze:



Eseguiamo il menu del client vaccinato, la cui funzione si occuperà di popolare i parametri di output, quali codice di tessera sanitaria e data di vaccinazione:

```
if(!run_vaccinated_menu(client_daytime, server_daytime, verification_code))
{
    /* Liberiamo la memoria precedentemente allocata dinamicamente nella memoria
    heap tramite una "@malloc" */
    free(server_daytime);
    free(client_daytime);
    free(verification_code);
    /* Chiusura del socket file descriptor connesso al server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_FAILURE);
}
```

Nel menu, verranno eseguite le funzioni di validazione del codice di tessera sanitaria e della data di vaccinazione. In particolare, la funzione di validazione codice di tessera sanitaria suddivide la stringa in tre sottostringhe ed effettua i seguenti controlli per verificarne la validità:

1. Codice di tessera sanitaria composto da soli valori numerici;
2. Codice di tessera sanitaria di dimensione 20;
3. Primi 5 caratteri del codice di tessera sanitaria equivalenti a 80380;
4. Successivi 5 caratteri del codice di tessera sanitaria equivalenti ad uno dei codici regione presenti nel file "tessera_sanitaria_codice_regioni".

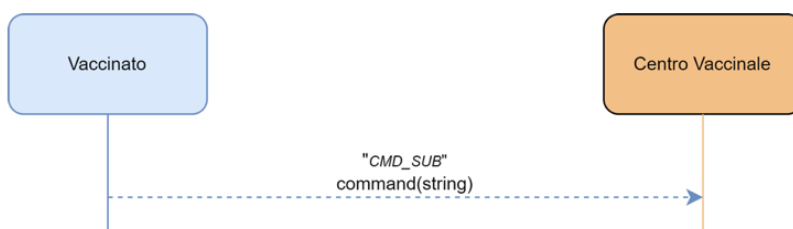
Mentre, per la funzione di validazione data, saranno effettuati i seguenti controlli:

1. Data nel formato dd/mm/yyyy;
2. Anno compreso nel range <2020..2022>;
3. Mese compreso nel range <1..12>;
4. Giorno compreso nel range <1..31>;
5. Se l'anno fornito è bisestile;
6. Se il mese inserito è febbraio;
7. Se il mese inserito è composto da 30 giorni.

Successivamente, eseguiamo la richiesta di iscrizione inviando il comando *CMD_SUB*:

```
FullWrite(client_file_descriptor, command_writer_buffer, CMD_BUFFER_LEN);
```

Come indicato dal protocollo livello applicazione:



Ed andiamo a criptare il codice di tessera sanitaria da inviare, come specificato nel protocollo livello applicazione:

```
xor_crypt(vaccinated_request_package.card_code, 14);
```

L'algoritmo di criptazione viene definito eseguendo uno xor bit a bit di tutti i caratteri che compongono la stringa del codice di tessera sanitaria, come mostrato di seguito:

```

char* xor_crypt(char* string_to_encrypt, int key)
{
    int i;
    for(i = 0; i < strlen(string_to_encrypt); i++)
    {
        /* Per l'operazione di criptazione prevediamo uno xor bit
        a bit di ogni carattere con la chiave passata come argomento */
        string_to_encrypt[i] ^= key;
    }

    /* Ritorniamo la stringa criptata/decriptata */
    return string_to_encrypt;
}

```

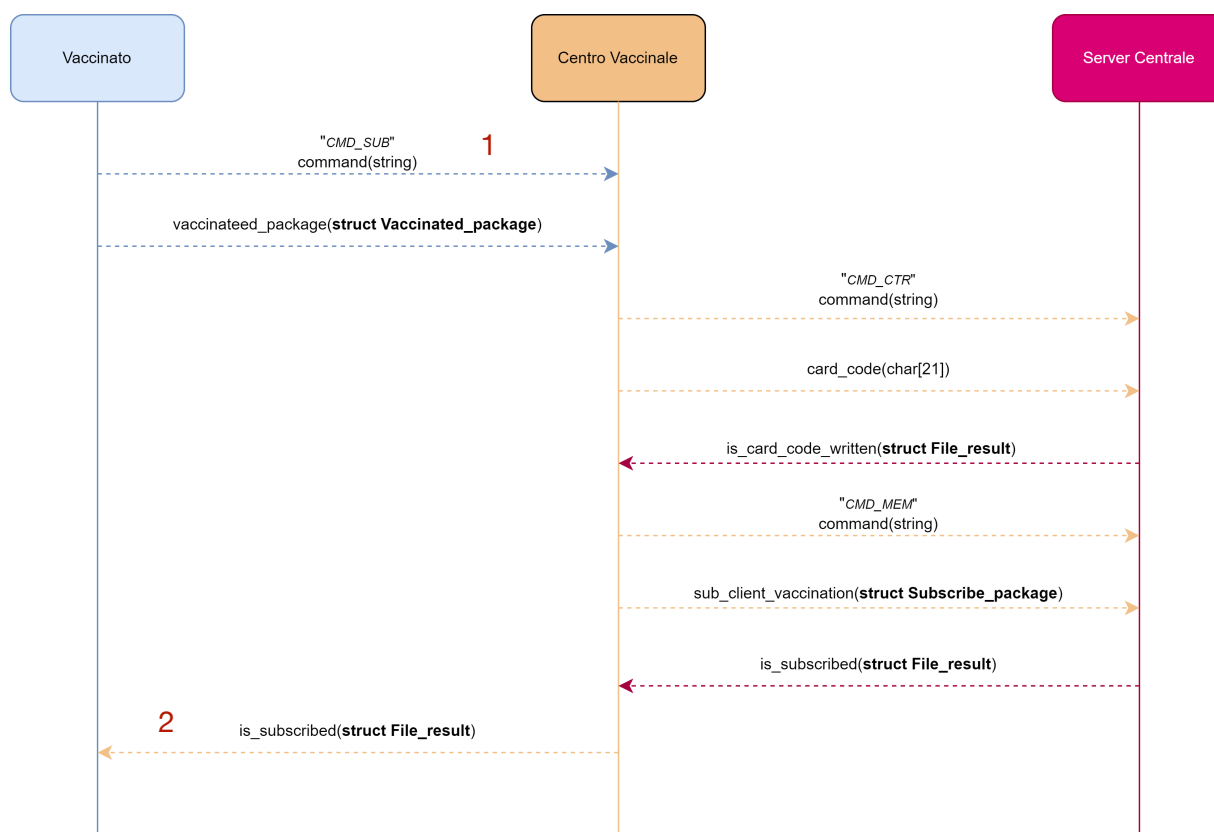
Successivamente, inviamo il pacchetto contenente la data di vaccinazione e il codice di tessera sanitaria dell'utente, al server centro vaccinale e restiamo in attesa di una risposta con la successiva FullRead:

```

FullWrite(client_file_descriptor, &vaccinated_request_package, sizeof(vaccinated_request_package)); 1
if(FullRead(client_file_descriptor, &is_green_pass_obtained, sizeof(File_result)) > 0)
{
    /* Liberiamo la memoria precedentemente allocata dinamicamente nella memoria heap tramite una 2
    "@malloc" */
    free(server_daytime);
    free(client_daytime);
    free(verification_code);
    /* Chiusura del socket file descriptor connesso al server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_FAILURE);
}

```

Come indicato dal protocollo livello applicazione:



Infine, aggiorniamo l'interfaccia, chiudiamo la connessione con il server ed interrompiamo l'esecuzione del programma.

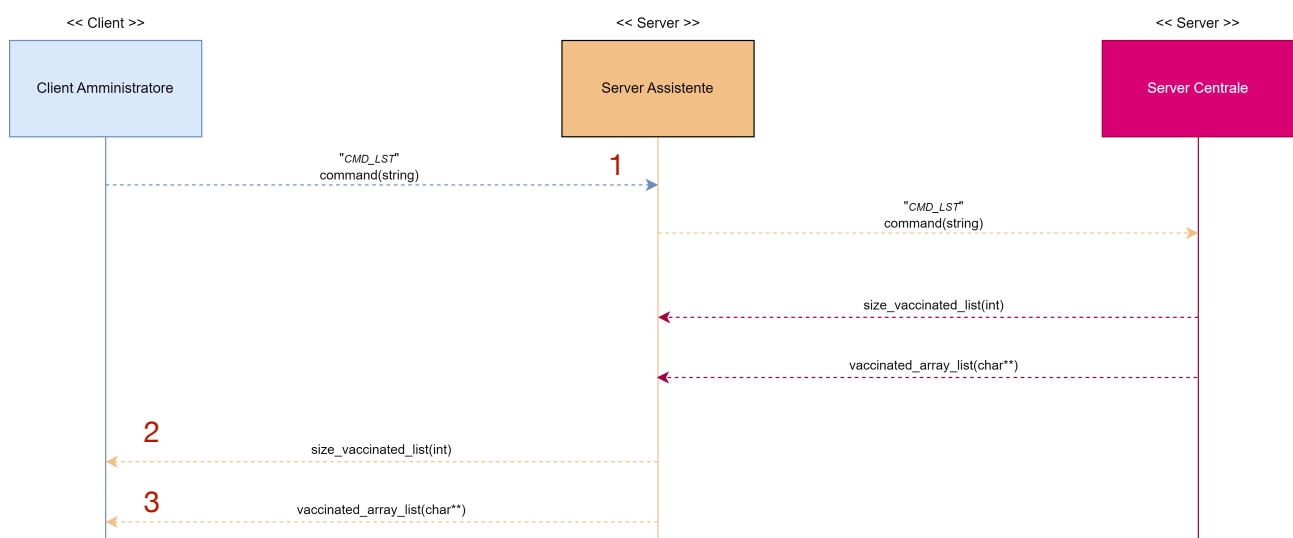
4.2 - Client Amministratore

Per il client amministratore si è deciso di implementare un DNS diretto. La socket, la valorizzazione delle strutture e la connessione con il server avvengono allo stesso modo del client vaccinato e revisore.

Successivamente, inviamo una richiesta *CMD_LST* per ottenere una lista di utenti da modificare:

```
FullWrite(client_file_descriptor, command_writer_buffer, CMD_BUFFER_LEN);
if(FullRead(client_file_descriptor, &size_list, sizeof(int)) > 0)
{
    fprintf(stderr, "Anomalia durante l'operazione del server\n");
    /* Chiusura del socket file descriptor connesso al server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_FAILURE);
}
if(FullRead(client_file_descriptor, code_list, size_list * 21) > 0)
{
    fprintf(stderr, "Errore di lettura\n");
    /* Chiusura del socket file descriptor connesso al server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_FAILURE);
}
```

Come indicato dal protocollo livello applicazione:



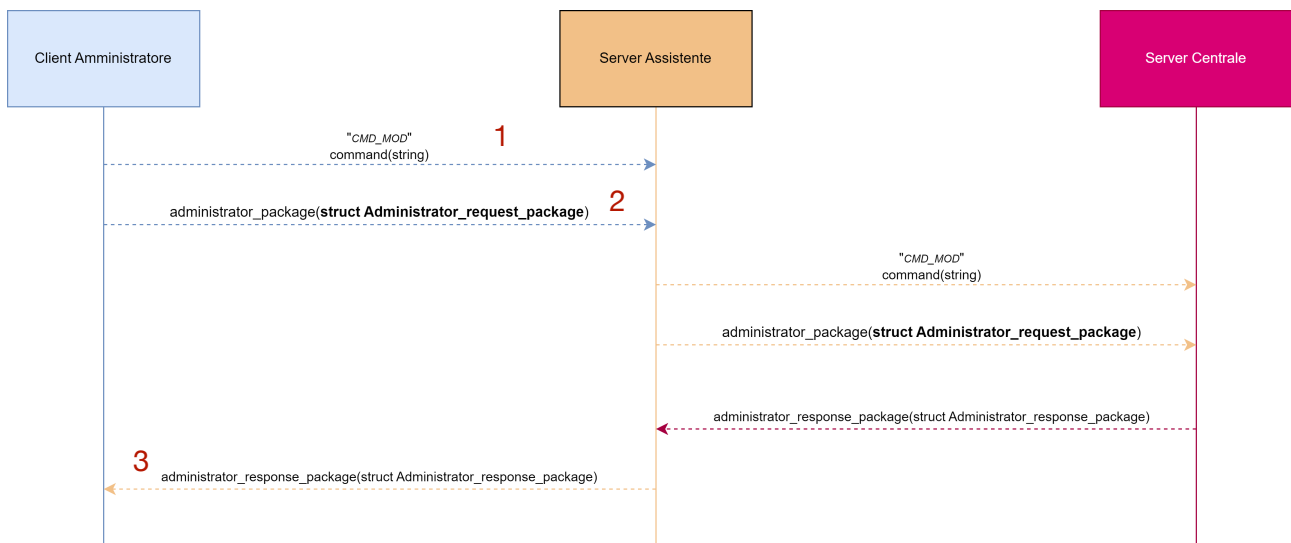
Eseguiamo il menu del client amministratore la cui funzione si occuperà di popolare il parametro di output, `administrator_package`:

```
if(!run_administrator_menu(&administrator_package, code_list, size_list
{)
    /* Chiusura del socket file descriptor connesso al server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_SUCCESS);
}
```

Inviando una richiesta `CMD_MOD` per modificare uno specifico utente iscritto alla piattaforma:

```
FullWrite(client_file_descriptor, command_writer_buffer, CMD_BUFFER_LEN); 1
FullWrite(client_file_descriptor, &administrator_package, sizeof(administrator_package)); 2
if(FullRead(client_file_descriptor, &administrator_response_package, sizeof(administrator_response_package)) > 0)
{
    fprintf(stderr, "Errore di lettura\n");
    /* Chiusura del socket file descriptor connesso al server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_FAILURE);
}
```

Come indicato dal protocollo livello applicazione:



Infine, aggiorniamo l'interfaccia, chiudiamo la connessione con il server ed interrompiamo l'esecuzione del programma.

4.3 - Client Revisore

Per il client revisore si è deciso di implementare un DNS diretto. La socket, la valorizzazione delle strutture e la connessione con il server avvengono allo stesso modo del client vaccinato e amministratore.

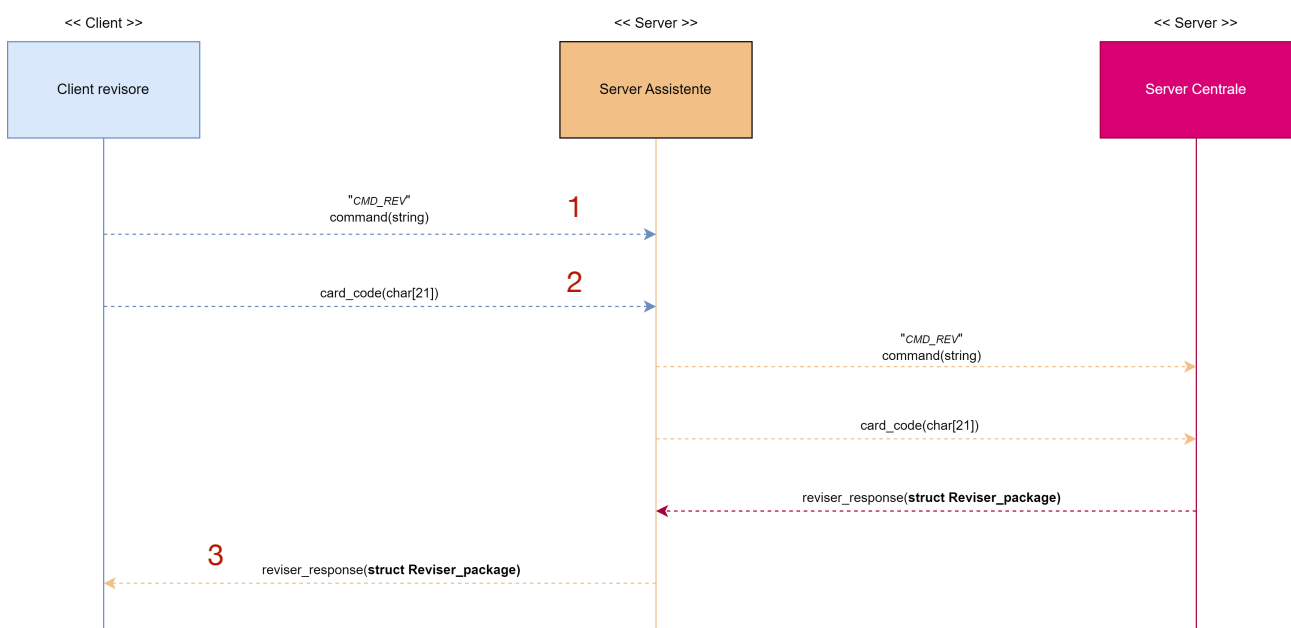
Eseguiamo il menu del client revisore la cui funzione si occuperà di popolare il parametro di output, codice di tessera sanitaria:

```
if(!run_reviser_menu(writer_buffer))
{
    /* Chiusura del socket file descriptor connesso al
    server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_FAILURE);
}
```

Successivamente, inviamo una richiesta `CMD_REV` per ottenere le informazioni del documento `Green Pass`:

```
FullWrite(client_file_descriptor, command_writer_buffer, CMD_BUFFER_LEN); 1
FullWrite(client_file_descriptor, xor_crypt(writer_buffer,14), 21); 2
if(FullRead(client_file_descriptor, &reviser_package, sizeof(reviser_package)) > 0)
{
    /* Chiusura del socket file descriptor connesso al server */
    close(client_file_descriptor);
    /* Terminiamo con successo il processo client */
    exit(EXIT_FAILURE);
}
```

Come indicato dal protocollo livello applicazione:



Infine, aggiorniamo l'interfaccia, chiudiamo la connessione con il server ed interrompiamo l'esecuzione del programma.

5.0 - Dettagli implementativi dei Server

In questa sezione andremo ad analizzare i dettagli implementativi dei server, per capire come si è deciso di strutturarli.

Tutti i server sono stati ingegnerizzati ed implementati sfruttando la logica dei thread. Tale scelta è stata presa, prendendo in considerazione la sicurezza del servizio offerto, in quanto, senza thread un client malintenzionato potrebbe comunicare uno stream di byte che comporterebbe il blocco del server e la conseguente interruzione del servizio. Invece, attraverso l'utilizzo dei thread, un client malintenzionato potrebbe bloccare solo l'esecuzione di un thread, permettendo agli altri client di essere serviti in qualsiasi caso.

Ogni server alla creazione di un thread prende in carico una funzione handler che si occuperà di servire la richiesta:

- Server centrale:

```
if((errno = pthread_create(&threads_id[i++], NULL, central_server_handler, &thread_arguments)) != 0)
{
    /* La seguente funzione produce un messaggio sullo standard error (file descriptor: 2) che descrive
    la natura dell'errore */
    perror("Thread creation error");
    break;
}
```

- Server centro vaccinale:

```
if((errno = pthread_create(&threads_id[i++], NULL, vaccination_center_handler, &thread_arguments)) != 0)
{
    /* La seguente funzione produce un messaggio sullo standard error (file descriptor: 2) che descrive la
    natura dell'errore */
    perror("Thread creation error");

    /* In caso di errore durante la creazione del thread interrompiamo il costrutto iterativo*/
    break;
}
```

- Server assistente:

```
if((errno = pthread_create(&threads_id[i++], NULL, assistant_server_handler, &thread_arguments)) != 0)
{
    /* La seguente funzione produce un messaggio sullo standard error (file descriptor: 2) che descrive
    la natura dell'errore */
    perror("Thread creation error");
    break;
}
```

Infatti nella libreria thread utility sono state definite tali funzioni:

```
void* central_server_handler(void*);
void* vaccination_center_handler(void*);
void* assistant_server_handler(void*);
```

Inoltre tutti i server, possono gestire molteplici richieste, pertanto per effettuare una qualsiasi operazione, sarà necessario inviare un comando per eseguire un set di istruzioni predefinite. Per tal motivo tutti i server sono stati implementati con la logica RPC (*Remote Procedure Call*), in modo da gestire all'interno dei thread le diverse tipologie di chiamate.

Ad esempio, nel server centro vaccinale vi troviamo i comandi:

- **CMD_DTM**: per la richiesta daytime;
- **CMD_SUB**: per la richiesta di iscrizione alla piattaforma;

```
if(!strcmp(command_reader_buffer, "CMD_DTM"))
{...}
else if(!strcmp(command_reader_buffer, "CMD_SUB"))
{...}
```

Nel server centrale vi troviamo i comandi:

- **CMD_CTR**: per la richiesta di verifica, utente già iscritto alla piattaforma;
- **CMD_MEM**: per la richiesta di iscrizione alla piattaforma;
- **CMD_REV**: per la richiesta di ottenimento delle informazioni relative al documento **Green Pass**;
- **CMD_LST**: per la richiesta di ottenimento della lista di utenti iscritti alla piattaforma;
- **CMD_MOD**: per la richiesta di modifica delle informazioni relative al documento **Green Pass** di uno specifico utente.

```
if(!strcmp(command_reader_buffer, "CMD_CTR"))
{...}
else if(!strcmp(command_reader_buffer, "CMD_MEM"))
{...}
else if(!strcmp(command_reader_buffer, "CMD_REV"))
{...}
else if(!strcmp(command_reader_buffer, "CMD_LST"))
{...}
else if(!strcmp(command_reader_buffer, "CMD_MOD"))
{...}
```

Nel server assistente vi troviamo i comandi:

- **CMD_REV**: per la richiesta di ottenimento delle informazioni relative al documento **Green Pass**;
- **CMD_LST**: per la richiesta di ottenimento della lista di utenti iscritti alla piattaforma;

- **CMD_MOD**: per la richiesta di modifica delle informazioni relative al documento **Green Pass** di uno specifico utente.

```
if (!strcmp(command_reader_buffer, "CMD_REV"))
{...}
else if(!strcmp(command_reader_buffer, "CMD_LST"))
{...}
else if(!strcmp(command_reader_buffer, "CMD_MOD"))
{...}
```

6.0 - Manuale utente

In questa sezione mostreremo le istruzioni per la compilazione, esecuzione ed l'interfaccia utente.

6.1 - Istruzioni per la compilazione ed esecuzione

Per facilitare la compilazione e l'esecuzione dei file sorgente è stato realizzato un makefile contenente i seguenti comandi:

- Compilazione dei server:

```
# Build server Centrale
build_central_server:
gcc central_server.c \
lib/sockets_utility.c \
lib/thread_utility.c \
lib/file_utility.c \
lib/date_utility.c \
lib/green_pass_utility.c \
lib/encryption_utility.c \
-o ./output/server_centrale

# Build server assistente
build_assistant_server:
gcc assistant_server.c \
lib/sockets_utility.c \
lib/thread_utility.c \
lib/file_utility.c \
lib/date_utility.c \
lib/green_pass_utility.c \
lib/encryption_utility.c \
-o ./output/server_assistente

# Build server centro vaccinale
build_vaccinated_server:
gcc vaccination_center_server.c \
lib/thread_utility.c \
lib/sockets_utility.c \
lib/file_utility.c \
lib/date_utility.c \
lib/green_pass_utility.c \
lib/encryption_utility.c \
-o ./output/centro_vaccinale
```

- Compilazione dei client:

```
Build client vaccinale
build_vaccinated_client:
    gcc vaccinated_client.c \
        lib/sockets_utility.c \
        lib/menu_utility.c \
        lib/date_utility.c \
        lib/code_verification.c \
        lib/digits_utility.c \
        lib/file_utility.c \
        lib/buffer_utility.c \
        lib/green_pass_utility.c \
        lib/thread_utility.c \
        lib/encryption_utility.c \
        -o ./output/client_vaccinale

# Build client revisore
build_reviser_client:
    gcc reviser_client.c \
        lib/sockets_utility.c \
        lib/menu_utility.c \
        lib/date_utility.c \
        lib/code_verification.c \
        lib/digits_utility.c \
        lib/file_utility.c \
        lib/buffer_utility.c \
        lib/green_pass_utility.c \
        lib/thread_utility.c \
        lib/encryption_utility.c \
        -o ./output/client_reviser

# Build client amministratore
build_administrator_client:
    gcc administrator_client.c \
        lib/sockets_utility.c \
        lib/menu_utility.c \
        lib/date_utility.c \
        lib/code_verification.c \
        lib/digits_utility.c \
        lib/file_utility.c \
        lib/buffer_utility.c \
        lib/green_pass_utility.c \
        lib/thread_utility.c \
        lib/encryption_utility.c \
        -o output/client_administrator
```

- Compilazione di tutti i client e server:

```
# Build di tutti i client e server
build: build_central_server \
        build_assistant_server \
        build_vaccinated_server \
        build_vaccinated_client \
        build_administrator_client \
        build_reviser_client
```

Esecuzione dei server e dei client:

```
# Run del server centrale
run_central_server:
    cd ./output &&\
        sudo ./server_centrale

# Run del server assistente
run_assistant_server:
    cd ./output &&\
        sudo ./server_assistente

# Run del server vaccinale
run_vaccinated_server:
    cd ./output &&\
        sudo ./centro_vaccinale
```

```
# Run del client vaccinale
run_vaccinated_client:
    cd ./output &&\
        ./client_vaccinale localhost

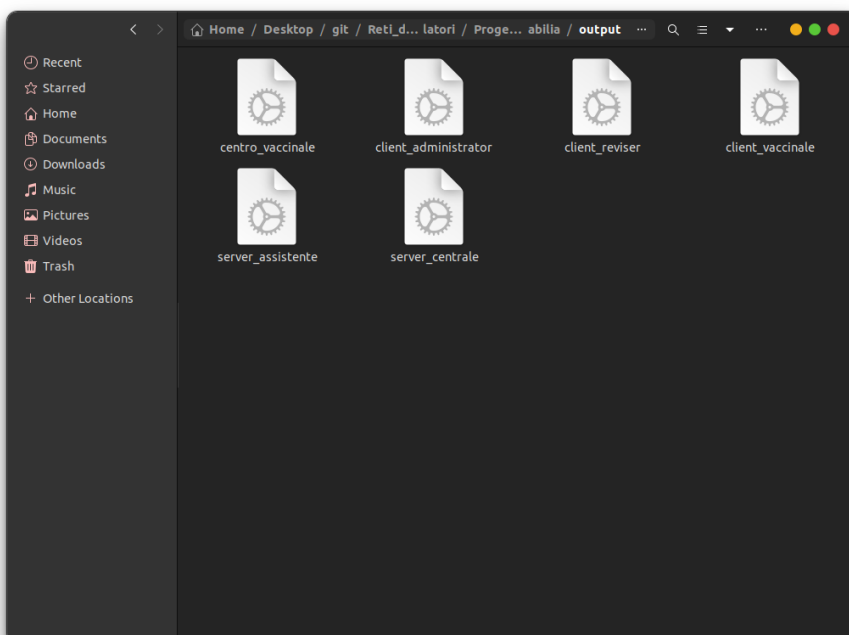
# Run del client revisore
run_reviser_client:
    cd ./output &&\
        ./client_reviser localhost

# Run del client amministratore
run_administrator_client:
    cd ./output &&\
        ./client_administrator localhost
```

Pertanto, per eseguire una compilazione, sarà necessario:

- Recarsi nella folder corrispondente al progetto consegnato:
Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia;
- Aprire un'istanza del terminale puntata alla folder indicata precedentemente;
- Eseguire il comando: **make** o **make build** per la compilazione del progetto, oppure eseguire singolarmente le build dei server e dei client, attraverso i seguenti comandi:
 1. **make build_central_server**
 2. **make build_assistant_server**
 3. **make build_vaccinated_server**
 4. **make build_vaccinated_client**
 5. **make build_reviser_client**
 6. **make build_administrator_client**

Una volta compilato il progetto, la cartella **output** sarà popolata con i file eseguibili, generati come mostrati di seguito:



Mentre, per eseguire tutti i client ed i server, sarà necessario:

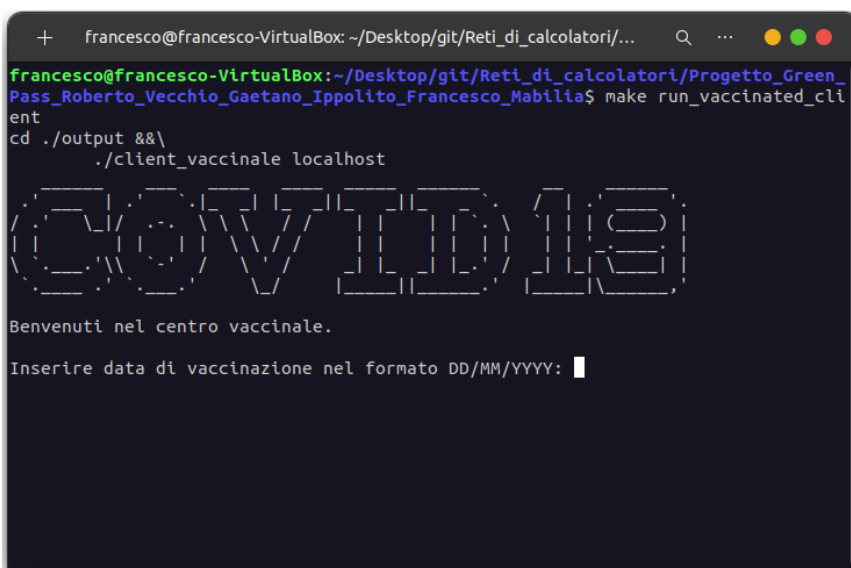
- Recarsi nella folder corrispondente al progetto consegnato:
Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia;
- Aprire sei istanze di terminale puntate alla folder indicata precedentemente;
- Eseguire per ogni terminale un solo comando:
 1. **make run_central_server**
 2. **make run_assistant_server**
 3. **make run_vaccinated_server**
 4. **make run_vaccinated_client**
 5. **make run_reviser_client**
 6. **make run_administrator_client**

6.2 - Istruzioni per l'interfaccia

Dopo aver eseguito tutti i server, l'esecuzione dei client e dei server si presenta come segue.

Client Vaccinato

Eseguito il client vaccinato avremo la seguente interfaccia:

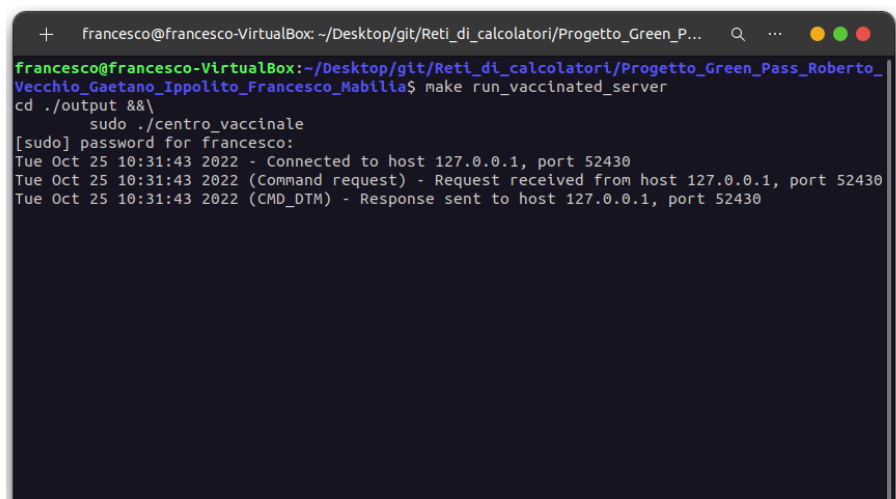


```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

GOVINDIS

Benvenuti nel centro vaccinale.
Inserire data di vaccinazione nel formato DD/MM/YYYY: 
```

Eseguito il client, il server centro vaccinale mostrerà, attraverso dei log, l'avvenuta connessione con il client e la richiesta/risposta daytime servita a quest'ultimo.



```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_server
cd ./output &&\
sudo ./centro_vaccinale
[sudo] password for francesco:
Tue Oct 25 10:31:43 2022 - Connected to host 127.0.0.1, port 52430
Tue Oct 25 10:31:43 2022 (Command request) - Request received from host 127.0.0.1, port 52430
Tue Oct 25 10:31:43 2022 (CMD_DTM) - Response sent to host 127.0.0.1, port 52430
```

A questo punto dell'esecuzione sarà necessario inserire una data di vaccinazione non inferiore ad un mese rispetto la data odierna e non superiore a quest'ultima. Inoltre sarà necessario inserire una data di vaccinazione che rispetti il formato indicato, ad esempio: 15/10/2022.

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progett...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COVID19

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 15/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
>
```

Se non si inserisce una data di vaccinazione valida, il client mostrerà un messaggio di errore:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COVID19

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 12-10-2022
Formato data invalida
make: *** [Makertile:10/: run_vaccinated_client] Error 1
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$
```

Dopo aver inserito la data di vaccinazione, sarà necessario inserire un codice di tessera sanitaria valido.

Un numero di tessera sanitaria è considerato valido se: è composto da sole cifre; le prime 5 cifre corrispondono a 80380, ovvero il numero di istituzione sanitaria specifica; le successive 5 cifre sono corrispondenti al codice regione e possono essere uguali ad uno di questi valori:

- 00010
- 00020
- 00030
- 00041
- 00042

- 00050
- 00060
- 00070
- 00080
- 00090
- 00100
- 00110
- 00120
- 00130
- 00140
- 00150
- 00160
- 00170
- 00180
- 00190
- 00200

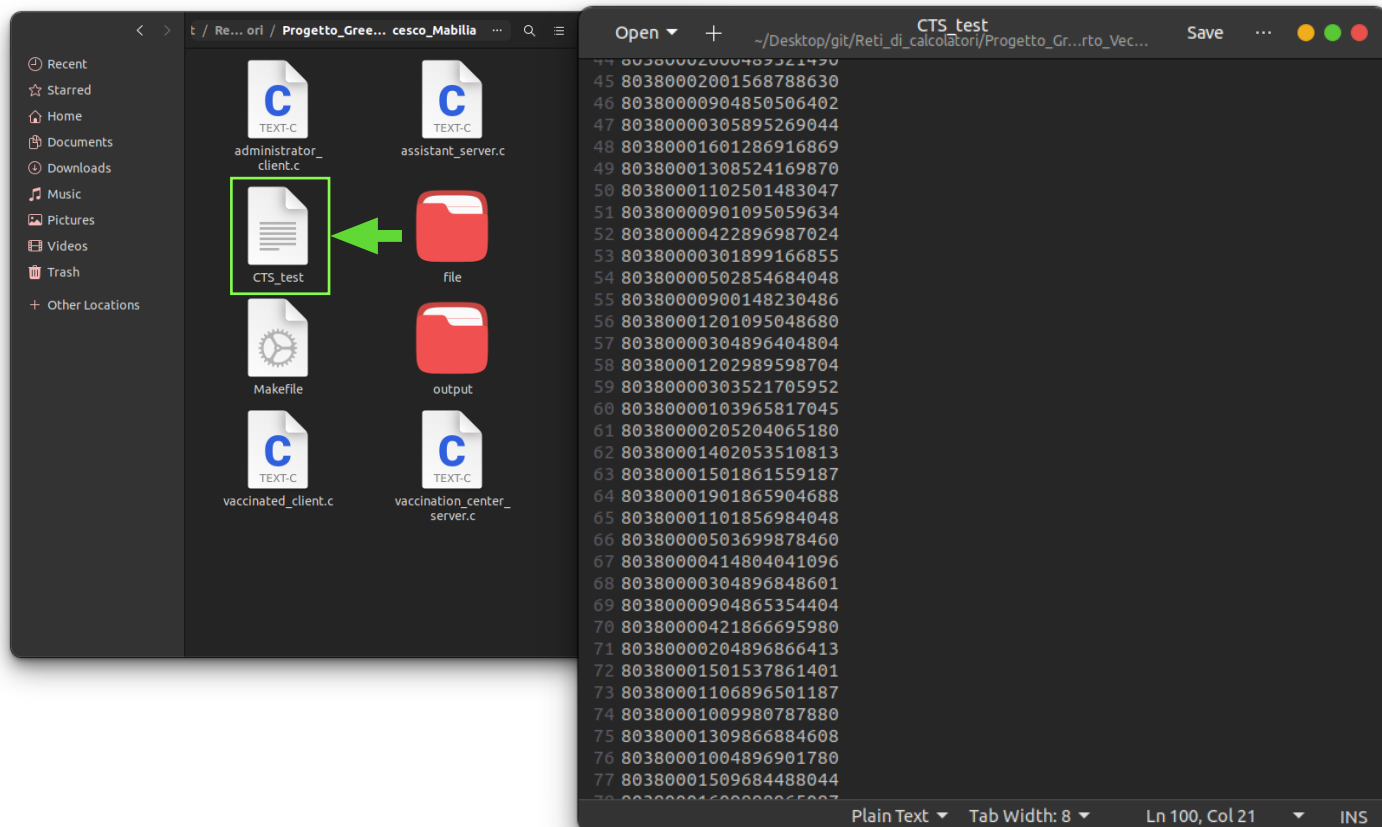
Le ultime 10 cifre possono essere composte da un qualsiasi valore numerico. Un esempio di codice valido è il seguente:

- 80380000103658942587

Mentre, sono un esempio di codice errato i seguenti:

- 81120000103658942587: prime 5 cifre diverse da 80380;
- 80380300003658942587: prime 5 cifre uguali a 80380, ma successive 5 cifre diverse da uno dei codici regione elencati precedentemente.

All'interno della folder consegnata, è stato creato un file contenente 100 codici di prova, chiamato *CTS_test*, l'utente può utilizzare tali codici per testare l'affidabilità del software, ma è comunque possibile utilizzare codici di tessera sanitaria inventati conformi alle regole precedentemente elencate.



Nel caso in cui si inserisse un codice di tessera sanitaria non valido, per una delle ragioni precedentemente elencate, avremo i seguenti messaggi di errore:

1. Il codice inserito non è composto da soli numeri:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COVID19

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 14/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
> 803800042974894063a
Il codice inserito non è composto da soli numeri
make: *** [makefile:107: run_vaccinated_client] Error 1
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$
```

2. Lunghezza codice errata

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COVID19

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 19/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
> 803800015012587459
Lunghezza codice errata
make: *** [makefile:107: run_vaccinated_client] Error 1
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$
```

3. Codice di verifica non valido

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COV1D19

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 15/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
> 80320001501472598634
Codice di verifica non valido
make: *** [Makefile:107: run_vaccinated_client] Error 1
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$
```

Invece, se inserito un codice di tessera sanitaria valido:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COV1D19

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 25/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
> 80380000414879607680
```


Successivamente, verrà chiesto di accettare i termini e condizioni sul trattamento dei dati:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COV1919

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 25/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
> 80380000414879607680
Accettare il trattamento dei dati:
1. Accetto
2. Non Accetto
> ☐
```

Nel caso in cui non si accetti, avremo il seguente messaggio di errore:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COV1919

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 25/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
> 80380000414879607680
Accettare il trattamento dei dati:
1. Accetto
2. Non Accetto
> 2
Termini e condizioni necessari per continuare ad utilizzare il software!
make: *** [Makefile:107: run_vaccinated_client] Error 1
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ ☐
```

Se l'utente non è già iscritto e vengono accettati i termini e condizioni, verrà notificato dell'avvenuta iscrizione:

- Client vaccinato

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

  O V I D I E
Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 25/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
> 80380000414879607680
Accettare il trattamento dei dati:
1. Accetto
2. Non Accetto
> 1
Caricato con successo
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$
```

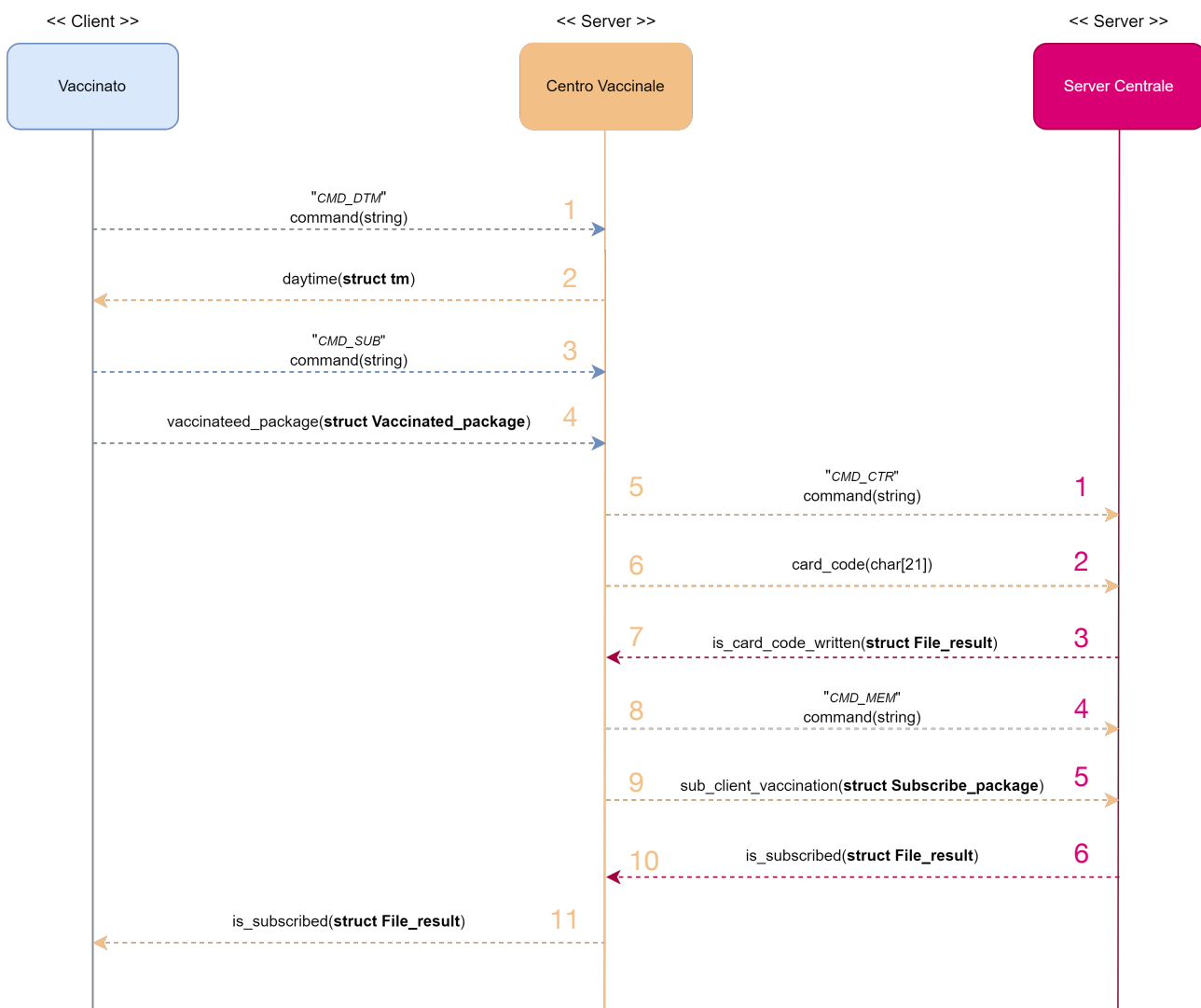
- Server centro vaccinale

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass...$ make run_vaccinated_server
cd ./output &&\
sudo ./centro_vaccinale
Tue Oct 25 11:21:53 2022 - Connected to host 127.0.0.1, port 33206
Tue Oct 25 11:21:53 2022 (Command request) - Request received from host 127.0.0.1, port 33206
Tue Oct 25 11:21:53 2022 (CMD_DTM) - Response sent to host 127.0.0.1, port 33206
Tue Oct 25 11:22:17 2022 (Command request) - Request received from host 127.0.0.1, port 33206
Tue Oct 25 11:22:17 2022 (CMD_SUB) - Request received from host 127.0.0.1, port 33206
Tue Oct 25 11:22:17 2022 (CMD SUB) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 11:22:17 2022 (CMD SUB) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 11:22:17 2022 (CMD SUB) - Response received from host 127.0.0.1, port 6464
Tue Oct 25 11:22:17 2022 (CMD SUB) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 11:22:17 2022 (CMD SUB) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 11:22:17 2022 (CMD SUB) - Response received from host 127.0.0.1, port 6464
Tue Oct 25 11:22:17 2022 (CMD SUB) - Response sent to host 127.0.0.1, port 33206
Tue Oct 25 11:22:17 2022 - Client disconnected: host 127.0.0.1, port 33206
```

- Server centrale

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecc...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetan
o_Ippolito_Francesco_Mabilia$ make run_central_server
cd ./output &&\
sudo ./server_centrale
[sudo] password for francesco:
Tue Oct 25 11:22:17 2022 - Connected to host 127.0.0.1, port 48560
Tue Oct 25 11:22:17 2022 (Command request) - Request received from host 127.0.0.1, port 48560
Tue Oct 25 11:22:17 2022 (CMD_CTR) - Request received from host 127.0.0.1, port 48560
Tue Oct 25 11:22:17 2022 (CMD_CTR) - Response sent to host 127.0.0.1, port 48560
Tue Oct 25 11:22:17 2022 (Command request) - Request received from host 127.0.0.1, port 48560
Tue Oct 25 11:22:17 2022 (CMD_MEM) - Request received from host 127.0.0.1, port 48560
Tue Oct 25 11:22:17 2022 (CMD_MEM) - Response sent to host 127.0.0.1, port 48560
Tue Oct 25 11:22:17 2022 - Client disconnected: host 127.0.0.1, port 48560
```

Corrispondenti alle comunicazioni descritte nel protocollo livello applicazione:



Nel caso in cui un utente sia già iscritto, avremo il seguente output per il client ed i server:

- Client vaccinato

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_client
cd ./output &&\
./client_vaccinale localhost

COVID19

Benvenuti nel centro vaccinale.

Inserire data di vaccinazione nel formato DD/MM/YYYY: 24/10/2022
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
> 80380001906904965003
Accettare il trattamento dei dati:
1. Accetto
2. Non Accetto
> 1
Errore nel caricamento
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$
```

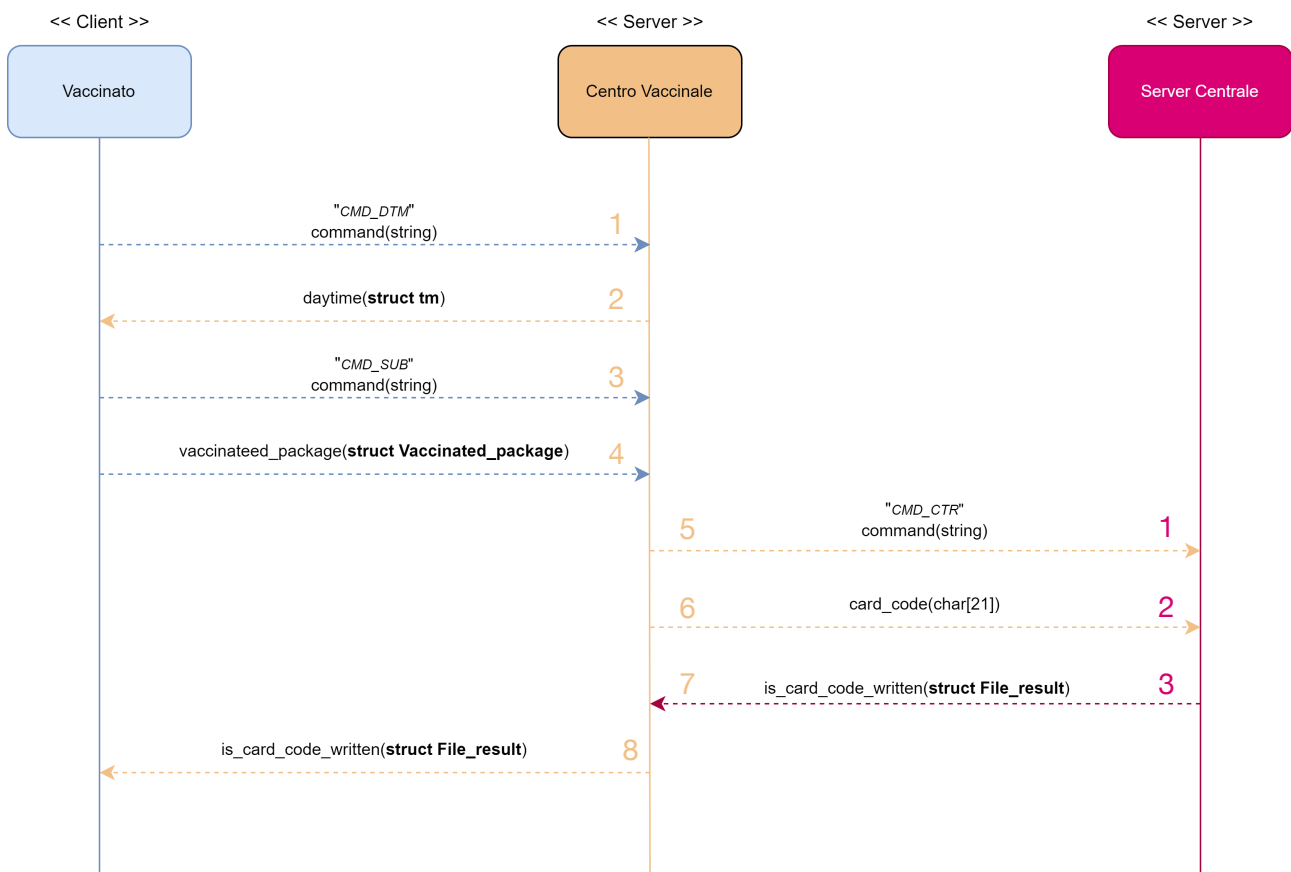
- Server centro vaccinale

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_vaccinated_server
cd ./output &&\
sudo ./centro_vaccinale
Tue Oct 25 11:26:06 2022 - Connected to host 127.0.0.1, port 36620
Tue Oct 25 11:26:06 2022 (Command request) - Request received from host 127.0.0.1, port 36620
Tue Oct 25 11:26:06 2022 (CMD DTM) - Response sent to host 127.0.0.1, port 36620
Tue Oct 25 11:26:18 2022 (Command request) - Request received from host 127.0.0.1, port 36620
Tue Oct 25 11:26:18 2022 (CMD SUB) - Request received from host 127.0.0.1, port 36620
Tue Oct 25 11:26:18 2022 (CMD SUB) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 11:26:18 2022 (CMD SUB) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 11:26:18 2022 (CMD SUB) - Response received from host 127.0.0.1, port 6464
Tue Oct 25 11:26:18 2022 (CMD SUB) - Response sent to host 127.0.0.1, port 36620
Tue Oct 25 11:26:18 2022 - Client disconnected: host 127.0.0.1, port 36620
```

- Server centrale

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecc...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_central_server
cd ./output &&\
sudo ./server_centrale
Tue Oct 25 11:26:18 2022 - Connected to host 127.0.0.1, port 46410
Tue Oct 25 11:26:18 2022 (Command request) - Request received from host 127.0.0.1, port 46410
Tue Oct 25 11:26:18 2022 (CMD_CTR) - Request received from host 127.0.0.1, port 46410
Tue Oct 25 11:26:18 2022 (CMD_CTR) - Response sent to host 127.0.0.1, port 46410
Tue Oct 25 11:26:18 2022 - Client disconnected: host 127.0.0.1, port 46410
```

Corrispondenti alle comunicazioni descritte nel protocollo livello applicazione:



Client Amministratore

Eseguito il client amministratore, nel caso in cui vi siano utenti iscritti, avremo la seguente interfaccia:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_administrator_client
cd ./output &&\
./client_administrator localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

=====
Indice    Codici Tessera Sanitaria
=====
1.        80380000103658942587
2.        803800002006547812935
3.        803800001502458647854
4.        803800001304785899547
5.        803800000702584858986
6.        80380000505855989887
7.        803800001804785485485
8.        80380000422559655221
9.        80380000411455415563
10.       80380000903633258749
11.       803800001106985554712
12.       803800001002144412563
13.       803800001909658741236
14.       80380000201596587415
15.       80380000103214569874
=====
```

Nel caso in cui non vi siano utenti iscritti, il client notificherà l'utente amministratore di tale condizione:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_administrator_client
cd ./output &&\
./client_administrator localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

Non ci sono elementi da aggiornare
```

Prima della stampa lista, il client effettua una richiesta di ottenimento della lista utenti al server assistente, pertanto, per il server assistente e centrale avremo il seguente output:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vec
chio_Gaetano_Ippolito_Francesco_Mabilia$ make run_assistant_server
cd ./output &&\
sudo ./server_assistente
Tue Oct 25 12:53:57 2022 - Connected to host 127.0.0.1, port 44374
Tue Oct 25 12:53:57 2022 (Command request) - Request received from host 127.0.0.1, port 44374
Tue Oct 25 12:53:57 2022 (CMD_LST) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 12:53:57 2022 (CMD_LST) - Response received from host 127.0.0.1, port 6464
Tue Oct 25 12:53:57 2022 (CMD_LST) - Response received from host 127.0.0.1, port 6464
Tue Oct 25 12:53:57 2022 (CMD_LST) - Response sent to host 127.0.0.1, port 44374
Tue Oct 25 12:53:57 2022 (CMD_LST) - Response sent to host 127.0.0.1, port 44374
```

```

francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecc...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetan
o_Ippolito_Francesco_Mabilia$ make run_central_server
cd ./output &&\
sudo ./server_centrale
Tue Oct 25 12:53:57 2022 - Connected to host 127.0.0.1, port 52160
Tue Oct 25 12:53:57 2022 (Command request) - Request received from host 127.0.0.1, port 52160
Tue Oct 25 12:53:57 2022 (CMD_LST) - Response sent to host 127.0.0.1, port 52160
Tue Oct 25 12:53:57 2022 (CMD_LST) - Response sent to host 127.0.0.1, port 52160
Tue Oct 25 12:53:57 2022 - Client disconnected: host 127.0.0.1, port 52160

```

Successivamente, verrà chiesto di selezionare un elemento della lista attraverso il corrispettivo indice:

```

francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_administrator_client
cd ./output &&\
./client_administrator localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

=====
Indice    Codici Tessera Sanitaria
=====
1.        80380000103658942587
2.        80380002006547812935
3.        80380001502458647854
4.        80380001304785899547
5.        80380000702584858986
6.        80380000505855989887
7.        80380001804785485485
8.        80380000422559655221
9.        80380000411455415563
10.       80380000903633258749
11.       80380001106985554712
12.       80380001002144412563
13.       80380001909658741236
14.       80380000201596587415
15.       80380000103214569874
=====

Scegliere l'indice di uno dei seguenti codici tessera sanitaria: 

```

Nel caso in cui si selezionasse un indice non esistente nella lista mostrata, verrà stampato a video il seguente messaggio di errore:

```

francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_administrator_client
cd ./output &&\
./client_administrator localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

=====
Indice    Codici Tessera Sanitaria
=====
1.        80380000103658942587
2.        80380002006547812935
3.        80380001502458647854
4.        80380001304785899547
5.        80380000702584858986
6.        80380000505855989887
7.        80380001804785485485
8.        80380000422559655221
9.        80380000411455415563
10.       80380000903633258749
11.       80380001106985554712
12.       80380001002144412563
13.       80380001909658741236
14.       80380000201596587415
15.       80380000103214569874
=====

Scegliere l'indice di uno dei seguenti codici tessera sanitaria: 16
Elemento selezionato non esistente

```


A seguire, il client richiederà di selezionare una delle due motivazioni di modifica mostrate a video (covid o guarigione):

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_administrator_client
cd ./output && \
./client_administrator localhost

  COVID19

Benvenuti sulla piattaforma Green Pass.

=====
Indice    Codici Tessera Sanitaria
=====
1.        80380000103658942587
2.        803800002006547812935
3.        803800001502458647854
4.        803800001304785899547
5.        80380000702584858986
6.        80380000505855989887
7.        803800001804785485485
8.        80380000422559655221
9.        80380000411455415563
10.       80380000903633258749
11.       803800001106985554712
12.       803800001002144412563
13.       803800001909658741236
14.       803800000201596587415
15.       80380000103214569874

=====

Scegliere l'indice di uno dei seguenti codici tessera sanitaria: 10
Scegliere il codice della motivazione:
1. Guarigione
2. Covid
> ☐
```

Nel caso in cui si selezionasse un indice di motivazione non presente a video, verrà stampato il seguente messaggio di errore:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_administrator_client
cd ./output && \
./client_administrator localhost

  COVID19

Benvenuti sulla piattaforma Green Pass.

=====
Indice    Codici Tessera Sanitaria
=====
1.        80380000103658942587
2.        803800002006547812935
3.        803800001502458647854
4.        803800001304785899547
5.        80380000702584858986
6.        80380000505855989887
7.        803800001804785485485
8.        80380000422559655221
9.        80380000411455415563
10.       80380000903633258749
11.       803800001106985554712
12.       803800001002144412563
13.       803800001909658741236
14.       803800000201596587415
15.       80380000103214569874

=====

Scegliere l'indice di uno dei seguenti codici tessera sanitaria: 10
Scegliere il codice della motivazione:
1. Guarigione
2. Covid
> 3
Scelta selezionata errata
```


Se la modifica dell'utente specificato, è possibile, allora verranno ritornate al client le informazioni aggiornate ed avremo la seguente interfaccia:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_administrator_client
cd ./output &&\
./client_administrator localhost

  COV I D 1 9

Benvenuti sulla piattaforma Green Pass.

=====
Indice    Codici Tessera Sanitaria
=====
1.        80380000103658942587
2.        80380002006547812935
3.        80380001502458647854
4.        80380001304785899547
5.        80380000702584858986
6.        80380000505855989887
7.        80380001804785485485
8.        80380000422559655221
9.        80380000411455415563
10.       80380000903633258749
11.       80380001106985554712
12.       80380001002144412563
13.       80380001909658741236
14.       80380000201596587415
15.       80380000103214569874
=====

Scegliere l'indice di uno dei seguenti codici tessera sanitaria: 10
Scegliere il codice della motivazione:
1. Guarigione
2. Covid
> 2
Le informazioni sono state aggiornate come segue:
=====
= Validità:           Green Pass non valido
=====
= Scadenza:          -/-/-
=====
= Motivazione:       Covid
=====
= Ultimo Aggiornamento: 25/10/2022
=====
```

Mentre i server produrranno i seguenti Log:

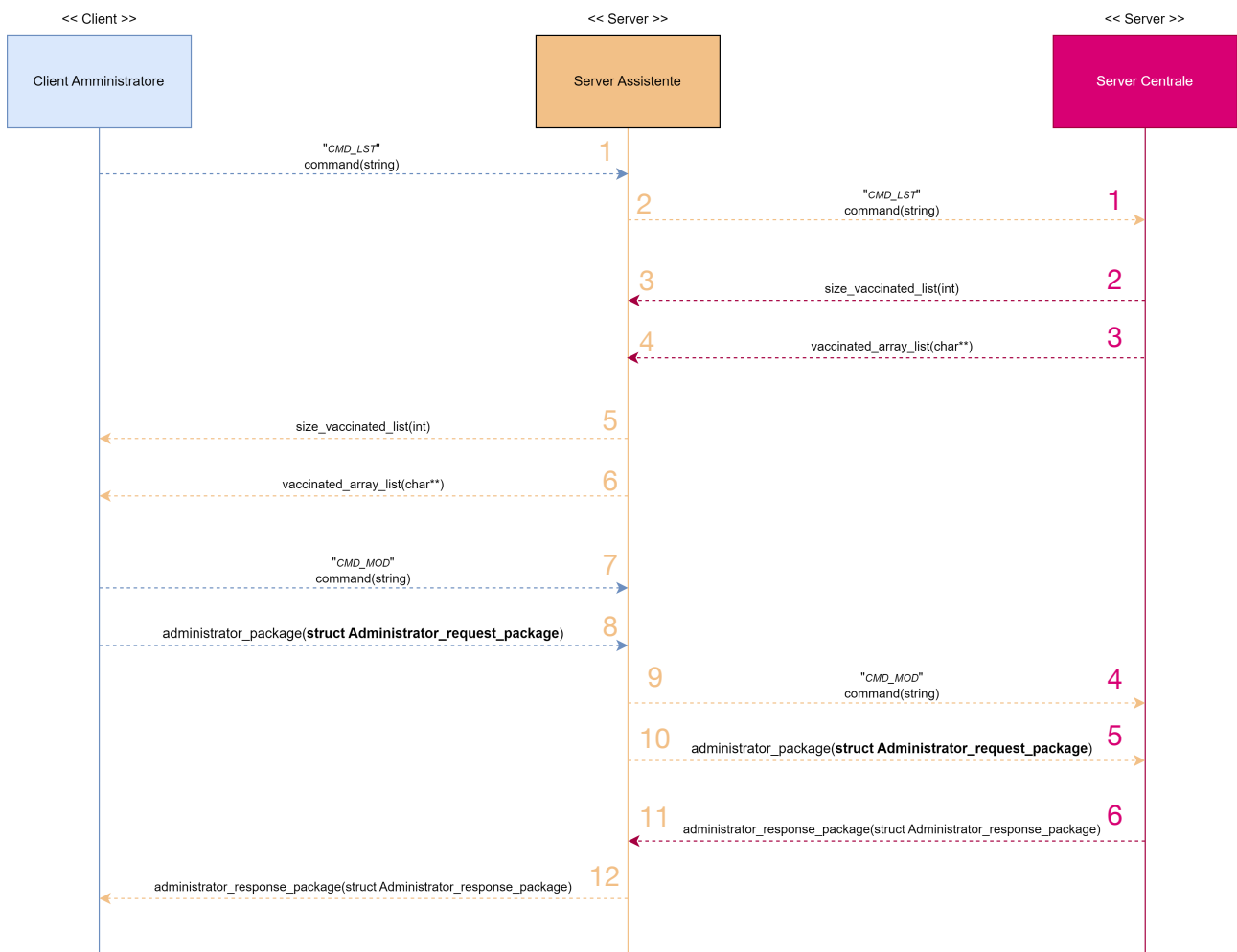
- Server assistente

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_assistant_server
cd ./output &&\
sudo ./server_assistente
Tue Oct 25 13:16:49 2022 - Connected to host 127.0.0.1, port 51218
Tue Oct 25 13:16:49 2022 (Command request) - Request received from host 127.0.0.1, port 51218
Tue Oct 25 13:16:49 2022 (CMD LST) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 13:16:49 2022 (CMD LST) - Response received from host 127.0.0.1, port 6464
Tue Oct 25 13:16:49 2022 (CMD LST) - Response received from host 127.0.0.1, port 6464
Tue Oct 25 13:16:49 2022 (CMD LST) - Response sent to host 127.0.0.1, port 51218
Tue Oct 25 13:16:49 2022 (CMD LST) - Response sent to host 127.0.0.1, port 51218
Tue Oct 25 13:16:54 2022 (Command request) - Request received from host 127.0.0.1, port 51218
Tue Oct 25 13:16:54 2022 (CMD MOD) - Request received from host 127.0.0.1, port 51218
Tue Oct 25 13:16:54 2022 (CMD MOD) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 13:16:54 2022 (CMD MOD) - Request sent to host 127.0.0.1, port 6464
Tue Oct 25 13:16:54 2022 (CMD MOD) - Response received from host 127.0.0.1, port 6464
Tue Oct 25 13:16:54 2022 (CMD MOD) - Response sent to host 127.0.0.1, port 51218
Tue Oct 25 13:16:54 2022 - Client disconnected: host 127.0.0.1, port 51218
```

- Server centrale

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecc...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecc...
o_Ippolito_Francesco_Mabilia$ make run_central_server
cd ./output &&\
sudo ./server_centrale
Tue Oct 25 13:16:49 2022 - Connected to host 127.0.0.1, port 55132
Tue Oct 25 13:16:49 2022 (Command request) - Request received from host 127.0.0.1, port 55132
Tue Oct 25 13:16:49 2022 (CMD_LST) - Response sent to host 127.0.0.1, port 55132
Tue Oct 25 13:16:49 2022 (CMD_LST) - Response sent to host 127.0.0.1, port 55132
Tue Oct 25 13:16:49 2022 - Client disconnected: host 127.0.0.1, port 55132
Tue Oct 25 13:16:54 2022 - Connected to host 127.0.0.1, port 46118
Tue Oct 25 13:16:54 2022 (Command request) - Request received from host 127.0.0.1, port 46118
Tue Oct 25 13:16:54 2022 (CMD_MOD) - Request received from host 127.0.0.1, port 46118
Tue Oct 25 13:16:54 2022 (CMD_MOD) - Response sent to host 127.0.0.1, port 46118
Tue Oct 25 13:16:54 2022 - Client disconnected: host 127.0.0.1, port 46118
```

Corrispondenti alle comunicazioni descritte nel protocollo livello applicazione:



Se la modifica dell'utente specificato, non è possibile, l'interfaccia verrà aggiornata come segue:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_administrator_client
cd ./output &&\
./client_administrator localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

=====
Indice    Codici Tessera Sanitaria
=====
1.        80380000103658942587
2.        80380002006547812935
3.        80380001502458647854
4.        80380001304785899547
5.        80380000702584858986
6.        80380000505855989887
7.        80380001804785485485
8.        80380000422559655221
9.        80380000411455415563
10.       80380000903633258749
11.       80380001106985554712
12.       80380001002144412563
13.       80380001909658741236
14.       80380000201596587415
15.       80380000103214569874
=====

Scegliere l'indice di uno dei seguenti codici tessera sanitaria: 10
Scegliere il codice della motivazione:
1. Guarigione
2. Covid
> 1
Impossibile aggiornare l'utente selezionato con i campi scelti
```

Client Revisore

Eseguito il client revisore avremo la seguente interfaccia:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_reviser_client
cd ./output &&\
./client_reviser localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
di cui si vuole conoscere la scadenza Green Pass: 
```

Per ottenere le informazioni sul documento **Green Pass**, sarà necessario inserire un codice di tessera sanitaria valido. Ricordiamo che le condizioni di validità per il codice di tessera sanitaria, sono equivalenti a quelle elencate per il client vaccinato. Anche in questo caso verranno gestite le eccezioni come nel caso del client vaccinato:

1. Il codice inserito non è composto da soli numeri:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_reviser_client
cd ./output &&\
./client_reviser localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
di cui si vuole conoscere la scadenza Green Pass: 8038000010365894258a
Il codice inserito non è composto da soli numeri
```

2. Lunghezza codice errata

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_reviser_client
cd ./output &&\
./client_reviser localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
di cui si vuole conoscere la scadenza Green Pass: 80380000103658942
Lunghezza codice errata
```

3. Codice di verifica non valido

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Proge...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass
_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_reviser_client
cd ./output &&\
./client_reviser localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
di cui si vuole conoscere la scadenza Green Pass: 80320001501234567899
Codice di verifica non valido
```

Invece, inserito il codice di tessera sanitaria valido, otterremo in risposta le informazioni inerenti al documento **Green Pass** dell'utente specificato:

- client:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_reviser_client
cd ./output &&\
./client_reviser localhost

COVID19

Benvenuti sulla piattaforma Green Pass.

Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera)
di cui si vuole conoscere la scadenza Green Pass: 80380000103658942587

Informazioni:
=====
= Validità:           Green Pass valido
=====
= Scadenza:           10/4/2023
=====
= Motivazione:        Vaccinazione
=====
= Ultimo Aggiornamento: 10/10/2022
=====
```

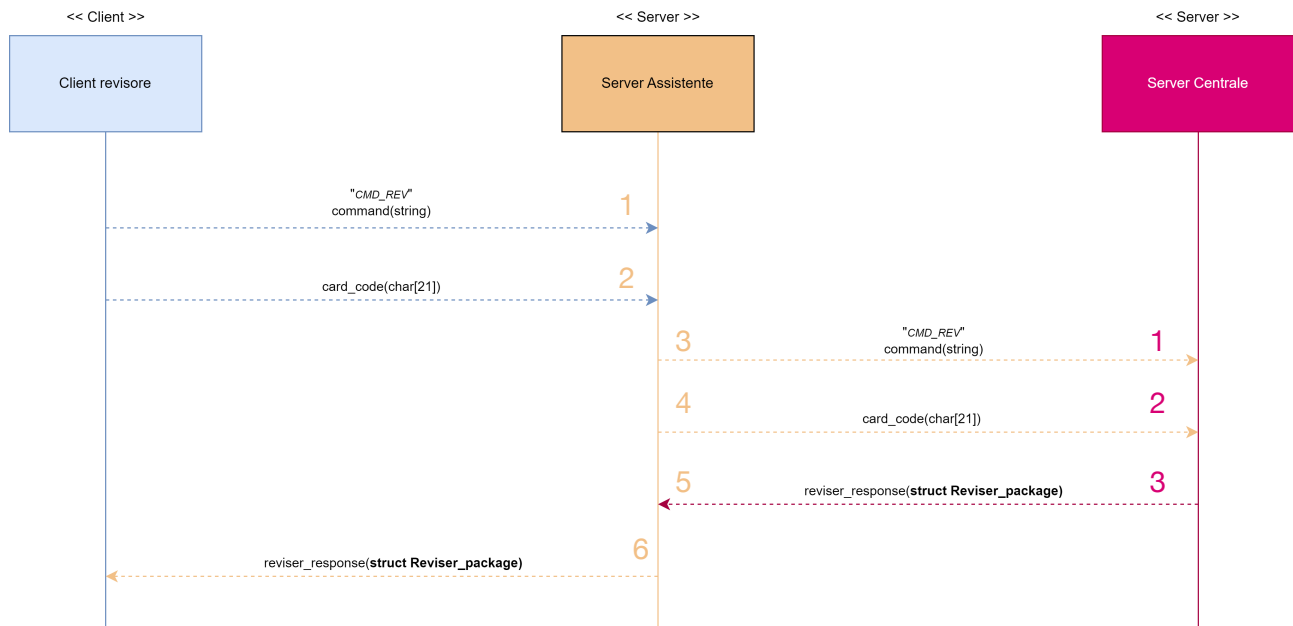
- Server assistente:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_assistant_server
cd ./output &&\
sudo ./server_assistente
[sudo] password for francesco:
Tue Oct 25 14:19:35 2022 - Connected to host 127.0.0.1, port 54496
Tue Oct 25 14:19:35 2022 (Command request) - Request received from host 127.0.0.1, port 54496 1
Tue Oct 25 14:19:35 2022 (CMD REV) - Request received from host 127.0.0.1, port 54496 2
Tue Oct 25 14:19:35 2022 (CMD REV) - Request sent to host 127.0.0.1, port 6464 3
Tue Oct 25 14:19:35 2022 (CMD REV) - Request sent to host 127.0.0.1, port 6464 4
Tue Oct 25 14:19:35 2022 (CMD REV) - Response received from host 127.0.0.1, port 6464 5
Tue Oct 25 14:19:35 2022 (CMD REV) - Response sent to host 127.0.0.1, port 54496 6
Tue Oct 25 14:19:35 2022 - Client disconnected: host 127.0.0.1, port 54496
```

- Server centrale:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_central_server
cd ./output &&\
sudo ./server_centrale
[sudo] password for francesco:
Tue Oct 25 14:19:35 2022 - Connected to host 127.0.0.1, port 42600
Tue Oct 25 14:19:35 2022 (Command request) - Request received from host 127.0.0.1, port 42600 1
Tue Oct 25 14:19:35 2022 (CMD REV) - Request received from host 127.0.0.1, port 42600 2
Tue Oct 25 14:19:35 2022 (CMD REV) - Response sent to host 127.0.0.1, port 42600 3
Tue Oct 25 14:19:35 2022 - Client disconnected: host 127.0.0.1, port 42600
```

Corrispondenti alle comunicazioni descritte nel protocollo livello applicazione:



Se si inserisce un codice di tessera sanitaria valido ma non iscritto alla piattaforma, si riceverà il seguente messaggio di errore:

```
francesco@francesco-VirtualBox: ~/Desktop/git/Reti_di_calcolatori/...
francesco@francesco-VirtualBox:~/Desktop/git/Reti_di_calcolatori/Progetto_Green_Pass_Roberto_Vecchio_Gaetano_Ippolito_Francesco_Mabilia$ make run_reviser_client
cd ./output &&\
./client_reviser localhost
COV0019
Benvenuti sulla piattaforma Green Pass.
Inserire codice tessera sanitaria (controllare il punto '8' sul retro della tessera) di cui si vuole conoscere la scadenza Green Pass: 80380001401254896350
Codice tessera sanitaria non esistente
```